Privacy-Preserving Genomic AI — Hardening & Novelty Roadmap

Eight-week plan for four people: from asserted guarantees to earned ones

Prepared as a technical review response · covers all issues raised in prior analysis · effort in person-days (pd)
Start with the **Plain-language overview** below if you are not the crypto lead; Section 0 onwards assumes familiarity.

Positioning: what kind of paper this is

This is not an advocacy paper arguing that functional encryption is the best tool for private inference. It almost certainly is not, for most workloads. It is an **exploratory evaluation of an under-examined design point**: FE is non-interactive, single-round, and uniquely permits the model owner to be offline during inference, and that combination has seen little systematic study in the genomic setting. The deliverable is a map — what this approach delivers, what it silently fails to deliver, what it costs, and when it is and is not the right choice.

That framing is worth adopting deliberately, because it changes what counts as success. Under an advocacy framing, discovering that the architecture leaks the model, the query and the result is a series of embarrassments to be patched quietly. Under an exploratory framing those discoveries **are the contribution** — they are the map. Report them as findings, report the costs of the fixes, and report honestly where the approach runs out of road. Everything in this document is written for the second framing.

How to read the box above — it is protective, not self-critical

Scaffolding for the team, not text for the paper. Do not paste any of it into the introduction — meta-commentary about how to read your own framing reads as defensiveness, which is the thing the framing exists to avoid.

The paragraph above describes *two* framings and rejects the first. It is easy to misread it as saying the project is weak. It says the opposite. Under an advocacy framing ("FE is the right tool, here is our system"), finding three leaks feels like three failures you would rather not mention. Under the exploratory framing, the same three discoveries **are your results**. Adopting the second framing is precisely what ensures nothing you find is embarrassing — the word in the paragraph above describes the framing you are *not* using.

"Almost certainly not the best tool for most workloads" is a statement about most workloads, not about yours. It is true of every tool in this space: Paillier is not best for most workloads, nor is FHE, nor secure hardware. Each is best in a narrow band. Saying so about your own is calibration, and calibration is what makes the rest of your claims believable.

Why FE is the right choice for this specific setting. One concrete reason, and it is sufficient: it is the only approach in which the model owner can be *offline*. Every MPC or additively-homomorphic scheme requires the hospital to be awake and answering for every single patient query. FE does not — issue a functional key once, then walk away. For a clinic–hospital–cloud deployment that is a real operational property, not a technicality (§0.7, row 0). This is the answer to "why not just use Paillier?", and it should appear in your introduction.

Why this makes the viva easier, not harder. Pre-registering your limitations means nobody can ambush you with them. The dangerous position is claiming FE is best and then being asked about secret sharing. Here you have already conceded the general point, so it cannot be used against you, and section 9 of the paper turns it into a contribution. Worked example —

Question	Answer
"Your cloud learns the diagnosis. Isn't that a failure of your design?"	"No — that is finding three. We identified it, we implemented three fixes, and we measured what each costs. It is in section 5."
"Why not use secure multi-party computation? It's faster."	"It is, for a single query. But it requires the model owner to participate in every inference. Ours does not have to be online at all — keys are provisioned once, then it can be switched off. That is the property we were evaluating."
"So functional encryption isn't actually the best approach?"	"Not for most workloads, no — and our decision guide says when to use it and when not to. Establishing that boundary was the point of the study."

Plain-language overview

This section explains the project and every proposed change without cryptographic vocabulary. It is written for a supervisor outside security, a viva panel, or a teammate who is not the crypto lead. Terms are defined in the glossary at the end. Everything here is restated formally later in the document.

P.1 The problem

A clinic has a patient's genetic data and wants a prediction from a trained model. *An accuracy point that matters: the model is a 33-way **tumour-type classifier**, not a cancer-risk predictor. Its input is expression data from a tumour biopsy, so the patient is already known to have a tumour; the output identifies which type. Calling it "cancer risk" is the first thing a clinical reviewer will reject, and it costs nothing to say correctly.* A hospital has a trained prediction model. Running the prediction requires bringing the data and the model together — and neither side wants to hand its side over. Sending raw genetic data to an outside computer is a serious privacy problem and in many places illegal. Sending the model out to every clinic means losing control of it, and if the model was trained on real patients' records, distributing it can expose those people too.

The usual workarounds each cost something. You can encrypt the data heavily and compute on it, but that is slow. You can have the two sides run an interactive back-and-forth protocol, but then the hospital must be awake and answering for every single patient query. You can use special secure hardware, but then you need that hardware.

P.2 What functional encryption does

Imagine a locked box and a special key. The key does not open the box. Instead, it reports *one specific fact* about what is inside — and nothing else. Hand someone a box of groceries and a key calibrated to "total weight," and they can tell you the weight without ever seeing what the groceries are.

That is functional encryption. The patient's genetic data goes in the box. The key is built from the hospital's model. Whoever holds both learns the prediction, and in principle nothing else. It has one property that the alternatives lack: **the hospital can issue the key once and then walk away.** With interactive methods the model owner must participate in every query; here it does not have to be online at all. That is the main reason this approach is worth studying, and the project should say so plainly.

The limitation is that the "one specific fact" has to be simple. The available schemes compute a weighted sum — multiply each gene value by a number and add them all up. That is enough for a linear model, and not obviously enough for anything more complicated. Section 10 is about pushing on that.

P.3 What the system currently does

Three separate computers, each deliberately given only part of the picture:

Party	Has	Job
Clinic	The patient's tumour expression data	Puts the data in the locked box. Receives the final score for the tumour type being tested.
Hospital	The trained model and the master secrets	Makes the special keys. Trusted by everyone.
Cloud	Neither — only a box and a key	Does the arithmetic. Not trusted.

The intended guarantee is that no single computer ever holds both the patient's data and the model in readable form.

P.4 What we found is wrong with it

Three leaks. The first is the serious one.

> **Leak 1 — the cloud can work out the entire model**
> The instructions for building a locked box are currently *public*. So the cloud does not have to wait for the clinic's box — it can build its own. It builds a box containing "1, 0, 0, 0, ..." and asks the key for the answer. Because the key computes a weighted sum, and only the first item is switched on, the answer it returns *is* the model's first number. Then it does the same with the second position, and the third. A couple of hundred repetitions later it has the complete model. This takes seconds, requires no cleverness, and needs no network access. Worse, the way key requests are currently handled means a dishonest clinic can do exactly the same thing.

Leak 2 — the cloud learns which cancer was suspected. Each cancer type uses a different number of genes, so the size of the box gives away which test was ordered — and in the newer design, so does which key the cloud picks. That reveals which tumour type or primary site is being investigated for a named patient, which in a workup for a tumour of unknown origin is arguably more sensitive than the gene values themselves.

Leak 3 — the cloud learns the answer. In the current code the cloud is the party that reads the result out of the box. So the untrusted computer learns the patient's score for the tumour type being tested, which is the single most sensitive output of the whole system. The project's own architecture description does not mention this.

P.5 The proposed fixes, in plain terms

Fix	In plain terms	What it costs
Switch to a secret box-making scheme *(function-hiding encryption)*	Change to a system where building a box requires a secret ingredient. The cloud does not have it, so it cannot build its own boxes, so it cannot run the guessing game of Leak 1.	A more complex, slower scheme. **It already exists in the library we use** and gives correct answers — but it cannot send data between computers, and it is far too slow (about 12 minutes per lookup). Fixing those two things is our main job.
Give the clinic a scrambled recipe	The clinic needs to build boxes, but giving it the hospital's master secret would let it forge keys too. Instead we give it a scrambled version of the recipe: enough to build boxes, not enough to work out the secret.	Box-building becomes noticeably slower. We measure how much.
Send the key in a sealed envelope	Rather than letting anyone request a key, the hospital seals it so only the cloud can open it, and the clinic just carries the envelope. A dishonest clinic cannot peek. **It does not fully solve Leak 1, though:** a dishonest clinic can still work out the model by asking many ordinary questions and reading the answers, because we send it answers by design. That path cannot be closed by encryption — only by limiting how many questions each clinic may ask.	Almost nothing. It also removes a whole component and two existing bugs.
Make every box the same size	Pad the unused space so the box size reveals nothing about which cancer was tested.	Bigger boxes, so slower. We plot exactly how much slower.
Hide the answer from the cloud	Three options. (A) The hospital adds a secret number to the answer that only the clinic knows how to subtract. (B) The cloud stops one step short and sends the half-finished result to the clinic to complete. (C) The cloud sends it to the hospital to complete instead.	Each option gives up something different — speed, or model secrecy, or the hospital being able to stay offline. Measuring that trade-off is one of our results.
Deliberate access for auditors	Regulators should be able to inspect the model. So we build a door for exactly that: an auditor gets the access that would let them reconstruct the model, it is logged, and the cloud never gets it.	About two days of work. It turns our most awkward finding into a feature.
Reshape the data first *(random feature maps)*	The system can only compute weighted sums. But if the clinic first passes the gene values through a fixed, publicly-known scrambling step, a weighted sum over the *scrambled* values behaves like a much richer, curved model over the original ones. We get a non-linear model without changing the encryption at all.	Needs many more values, so bigger boxes and more time. Finding the point where it becomes impractical is the experiment.

P.6 What we will actually produce

- **Working code** that makes an existing but unusable Python implementation actually usable — it cannot currently send encrypted data between computers, and we fix that. Useful to other researchers regardless of what our paper says.
- **Several demonstrated attacks** on the obvious way of building this kind of system, with measurements: how fast, how many attempts, how complete.
- **Fixes for each, with their costs measured** rather than asserted.
- **An honest map**: which kinds of medical model this approach can handle, which it cannot, and where it stops being practical. Including the parts where the answer is "do not use this."
- **Everything reproducible**: training script, pinned versions, tests, one command to start the whole system.

P.7 Glossary

Term	Meaning
Plaintext / ciphertext	Readable data / encrypted data. The "box" above is a ciphertext.
Inner product	Multiply matching pairs of numbers from two lists and add up the results. The only operation these schemes support, and exactly what a linear model needs.
Functional encryption (FE)	Encryption where a special key reveals one computed fact about the data instead of the data itself.
IPFE	Functional encryption where the "one fact" is an inner product. What the project uses now.
Function-hiding (FHIPE)	A stronger version where the special key also keeps the model's numbers secret. Requires that box-building be secret. Already implemented in PyMIFE (`mife/single/fhiding/ddh.py`) and verified correct — but not serializable and not fast enough, which is what we fix (§2.1).
Functional key (`sk_y`)	The "special key," built from the model's weights.
Master public / secret key	The public box-building instructions, and the hospital's master secret used to make functional keys.
KGC	Key Generation Centre — the trusted party that makes keys. The hospital, here.
Quantization	These schemes only handle whole numbers, so decimal values get multiplied up and rounded. Doing this carelessly loses accuracy, which is one of the bugs we found.
Chosen-plaintext attack	An attack where the adversary encrypts data of its own choosing to learn something. Exactly Leak 1.
Discrete logarithm / BSGS	The final step of decryption is a search for the answer among possible values. Its cost depends on how wide a range you have to search, which is why we profile the realistic range.
Pairing / bilinear group	The mathematical machinery the stronger scheme is built on. Choosing a library that supports it is our main technical risk.
FHE / MPC / TEE	The three main alternatives: compute-on-encrypted-data (slow), interactive protocols (model owner must stay online), and secure hardware (needs the hardware).
MIFE	A version where several organisations each encrypt their own data and the cloud combines them. Listed as future work; possibly cheaper to reach than we assumed.
Model extraction	Reconstructing a model from the answers it gives. Leak 1.
SHAP / XAI	Methods that explain which inputs drove a prediction. We propose doing this per biological pathway rather than per gene, which is both safer and more useful clinically.

0. What "protecting the model" is actually for

You were right to push on this. But the instinct has a consequence worth seeing before you commit, and then a resolution that is better than either horn of the dilemma.

0.1 The finding that forces all of this

Restating the load-bearing result, because everything downstream depends on it and it is easy to lose sight of. Damgård IPFE is a **public-key** scheme. Your Cloud holds `mpk` — you relay it in the payload, and it is fetchable from `/public_key` in any case — and it also receives `sk_y`. Anyone holding both can run this:

```
for i in range(k):
    ct       = FeDamgard.encrypt(unit_vector(i, k), mpk)
    y_hat[i] = FeDamgard.decrypt(ct, mpk, sk_y, bound)   # == y[i], exactly
```

`k` local operations, no network traffic, no cryptanalysis, and the untrusted node holds your complete quantized weight vector. Your weights are small integers so each search resolves instantly. Add `/model_metadata`, which hands over the active gene indices on request, and the extractor has the entire Lasso model: which genes, and what coefficient each carries.

This is not a bug in your code. It is a **theorem**, and it should be scoped precisely or a cryptographer will ping you for it: function hiding is only definable in the secret-key setting, because a public-key scheme necessarily gives the adversary oracle access to the function — it can encrypt anything and decrypt the result. Whether that oracle determines the function depends on the function class. *For inner products it determines it exactly, in `n` queries.* So the conclusion holds here with no wiggle room, but state it for the inner-product class rather than for functional encryption in general. Giving `sk_y` to a party that holds `mpk` is cryptographically equivalent to giving that party `y` in plaintext. Your Section 6.2 treats function hiding as a formalism that would make the claim "rigorous rather than heuristic." The claim is not heuristic; it is false, and the architecture table row asserting the Cloud does not hold ML weights needs to come out.

It is currently worse than Cloud-only, too. `/fetch_key` is unauthenticated and the *Doctor* generates the session ID, so a malicious clinician calls `/prep_key`, then calls `/fetch_key` itself, and runs the same extraction — looping over all 33 indices

to exfiltrate the whole model from a clinic terminal.

Secret-key FHIPE closes the oracle by making encryption require a secret. But it immediately raises the question that turns out to be the real intellectual content of the project:

The custody question
Under FHIPE, *somebody* must still be able to encrypt, and that somebody is the Doctor. If an encryption capability and a functional key ever land in the same pair of hands, the extraction attack returns immediately and unchanged. Model confidentiality in a deployed FE system is therefore **not a property of the cipher** — it is a property of how capabilities are partitioned across nodes, and it can only be stated per-coalition, never absolutely.

Correction: the condition is output access, not key possession

Stating the custody condition as "encryption capability plus a functional key" is **wrong**, and getting it wrong produces a false row in the coalition table (§5.3). The actual condition for extraction is **the ability to encrypt chosen inputs plus the ability to observe the corresponding outputs**. The Cloud satisfies it by holding `sk_y` and decrypting for itself. The *Doctor* satisfies it too — not by holding any key, but because the system returns scores to it by design. It submits `e_1, ..., e_k` through the ordinary inference path and reads the weights off the replies. Design A's blinding does not help, since the Doctor is told `rho` so that it can unblind.

This is not specific to functional encryption. Any prediction service leaks a linear model in roughly `n` queries (Tramèr et al., 2016 — cite it). Model confidentiality against the party that receives the predictions is **impossible by construction**, and no cryptography in any family will change that.

What capability separation actually buys, stated correctly: it does not prevent extraction, it changes its *cost and visibility*. Under the current design the Cloud extracts offline, instantly, for free, and undetectably — no interaction with anyone, nothing in any log. Under FHIPE the Cloud cannot extract at all, and the residual path through the Doctor requires `n` queries that are online, authenticated, logged, rate-limitable, billable — **and indistinguishable from legitimate clinical use**, for the reason below. Converting a free invisible attack into an expensive visible one is a real and defensible result. Claiming you eliminated the attack is not.

The sharper version: extraction is not an attack, it is a consequence of use

Recovering `y` exactly requires `n` linearly independent pairs `(x, <x,y>)`. It does **not** require chosen inputs, unit vectors, or any deviation from protocol — only a spanning set, and real patient vectors are generically linearly independent. So a clinic that legitimately tests `n` patients for one tumour type holds `n` equations in `n` unknowns and solves for the exact quantized weight vector by linear algebra, using only results it was entitled to receive. With 20–200 active genes per cancer, that is 20–200 patients: weeks of ordinary operation for the rarer cancers.

The consequence is that **no setting of a query budget separates clinical use from extraction, because they are the same queries.** A cap high enough to be useful is high enough to leak the model; a cap low enough to protect it makes the service pointless. The averaging attack noted later is a special case of this and understates it — the general attack needs no repetition at all, just more patients than parameters.

The result to claim instead — confidentiality as a lifetime
This is an upgrade on the claim it replaces, not a retreat. State it as:

> *Model confidentiality in outsourced linear inference has a lifetime measured in patients. A clinic serving `n` patients for a given model holds enough information to recover that model exactly, using only results it is entitled to. Dimension extends the lifetime linearly. Perturbation at noise scale `sigma` extends it by a measurable constant. Query budgets bound the rate; the audit log makes recovery attributable. None of them prevent it, and no cryptography in this family can.*

Provable, quantifiable, and to our knowledge unstated for FE-based inference. It also fixes the shape of your central experiment: plot **patients-until-recovery** against noise scale and against diagnostic-accuracy loss, on one axis, per cancer. The honest conclusion is then visible in a single figure, and the rare cancers — smallest `n`, shortest lifetime — are visibly the exposed ones.

Downstream consequence: what each control actually buys

Route 4 in §2.5 lists per-clinic budgets and noise as the consolation prize if FHIPE fails. They belong in the core plan whatever happens at Gate 1 — but be precise about what each one does, because none of them prevent anything:

Control	What it buys	What it does not buy
Dimension (§1, F4 decision)	Lifetime, **linearly in** `n`	The only lever that scales. Padding to the global union raises the worst-case model from ~20 patients to ~800 — roughly 40× — and protects precisely the rare cancers that are currently most exposed. This is why the union computation is the highest-value item in Week 1.
Perturbation	A measurable constant factor	Recovery from `m` noisy observations converges at `sigma·sqrt(n/m)`, so noise buys patient count, not a barrier. Also defeated by mask rescaling (§3 box).
Query budget	Rate	Not prevention. Cannot distinguish extraction from use.
Audit log	Attribution	Tells you who and when, after the fact.

This is what your three-node architecture is actually *for*, and it is the answer to the objection that a 200-dimension dot product does not need a cloud. The Cloud is not there for compute. **The Cloud is the party deliberately denied encryption capability, so that it is safe to hand it a functional key.** The Doctor can encrypt and must still never receive `sk_y` — though as the correction below makes clear, withholding the key is necessary and *not* sufficient, since the Doctor is sent the outputs anyway. The Hospital can do both and is trusted.

Two consequences to state plainly rather than bury. **Doctor–Cloud collusion breaks model confidentiality and no cryptography in this family will fix that** — the coalition trivially satisfies the condition below. And more importantly, **collusion is not required**: a malicious Doctor acting alone, through the legitimate inference path, is already sufficient. Handle both with rate limiting, query budgets, noise, the audit log and contractual separation, and say in the paper that that is what you are doing.

0.2 If the weights are public, this project has no reason to exist

If the Doctor holds the coefficients, they run inference locally in microseconds. Patient data never leaves the clinic — privacy solved, trivially, with no cryptography at all. There is then no Cloud, no functional encryption, and no paper. "Make the model public" is therefore not a modification to your architecture; it is a decision to abandon it. State that to yourselves plainly so the choice is made deliberately rather than by drift.

0.3 "Inspectable" and "coefficients withheld" are not opposites

The dichotomy you are assuming is false. A model can be *fully* auditable while its coefficient magnitudes stay protected. What clinicians and regulators actually need to interrogate is: training-cohort composition and provenance; the feature-selection methodology; performance on held-out *and* external cohorts, stratified by demographic subgroup; calibration; documented failure modes; version history. None of that requires publishing 33 × 20,000 floating-point numbers.

So publish all of it — the training code, the gene sets, the intercepts, the validation results, the subgroup analysis. Withhold only the magnitudes. Transparency is about *accountability for how the model behaves*, not about disclosure of parameter values, and conflating the two is a common error worth a paragraph in your paper.

0.4 Correction: where the confidentiality requirement actually comes from

You are right and my previous framing was wrong for your setup. TCGA is a public, consented research resource; sample barcodes and expression values are already published. Membership inference against a cohort whose membership is *already a matter of public record* is not a privacy harm, and model inversion recovering expression values that anyone can download is not one either. Do not build the paper on that.

The correct framing is the standard one for privacy work, and it is stronger because it matches the deployment you are actually proposing:

TCGA is a reproducible surrogate, not the deployment

A hospital deploying this does not serve a model trained on TCGA — if it did, every clinic would just download TCGA and fit their own. It serves a model trained on *its own* patient cohort, or a consortium's pooled non-public cohort, which is precisely what makes the model worth centralizing. The confidentiality requirement derives from **that** setting. You prototype on TCGA because it is public and reproducible, which is exactly why privacy-attack papers use public benchmarks: you cannot run experiments on data you are not permitted to hold. Say this explicitly in the paper — one paragraph, in the threat model — and the position is airtight. Leave it implicit and a reviewer will make the objection you just made.

Consequence for §5.2. The membership-inference experiment survives, but demoted: it is a *methodological measurement on a public surrogate* that bounds what leakage would look like on a private cohort of comparable dimensionality, not a demonstration of harm to TCGA participants. That is a legitimate and modest contribution. It is no longer the spine of the paper. Treat it as optional, and cut it before anything in §0.5.

0.5 The spine: three inference-side properties, and the one you are missing

Your focus is inference, not training. Good — that is where your architecture actually earns its keep, and it gives you a cleaner paper than either the IP framing or the training-cohort framing. At inference time there are exactly three things that can leak to the untrusted Cloud:

Property	What it protects	Your status	Notes
Data privacy	The patient's expression vector	**Held**	This is what IPFE gives you and it works. Already your strongest claim.
Query privacy	Which tumour type is being tested	**Broken**	Leaked through key selection under the decided architecture, and through ciphertext dimension before it (§5.4a). Arguably *more* sensitive than the expression values: it reveals which primary site or tumour type is under investigation for a named patient at a named time — the live question in a tumour-of-unknown-primary workup. Promote it from footnote to first-class result.
Result privacy	The patient's score for the type tested	**Broken**	See below; addressed at 6–7 pd in §5.4b under all-keys. Your own architecture table does not mention it.

> **The gap: your Cloud learns the diagnosis.** In `cloud_node.py` the Cloud performs `FeDamgard.decrypt()` itself, prints the result, and returns it. So the untrusted node obtains the raw inner product — a monotone function of the patient's cancer risk, and given the intercept (a single number, and one you extract into a file you describe as "harmless"), the risk percentage exactly. Your Section 3.1 table asserts the Cloud holds no patient data in plaintext. The expression *vector* is indeed protected; the clinically meaningful *output* is not. Combined with the query-privacy leak above, the Cloud learns "this patient scores 88% for breast primary," which is most of what there was to protect. This is squarely in your area of focus, it is a real hole, and subject to the Week-1 literature check, we are not aware of it having been written up for FE-based inference.

Two fixes, and a trade-off between them

Design A — Hospital-blinded score. Extend the vector by one slot: the Doctor encrypts `[x, 1]`, the Hospital issues a key for `[y, rho]` with fresh random `rho`, and sends `rho` to the Doctor only (sealed under the Doctor's public key, alongside the sealed key delivery of §2.8). The Cloud computes `<x,y> + rho`, which is uninformative; the Doctor subtracts. Statistical leakage is roughly `S / R` where `S` is the score range and `R` the blinding range, so `k` bits of blinding needs `R = 2^k · S`.

The cost is the interesting part: the Cloud's discrete-log search must now cover `R` rather than `S`, so decryption slows by roughly `2^(k/2)`. Result privacy is therefore not free — it trades directly against latency, and you can plot that curve. Combined with Finding 1 (§2.4), where the search cannot be amortized under FHIPE, this is a genuinely useful measurement.

Design B — split decryption. The Cloud does the pairings and returns the two GT elements `D1, D2` without solving the discrete log; the Doctor solves it locally over the small range `S`. The Cloud never learns the score, there is no blinding overhead, and the Cloud still performs the dominant cost (the `n` pairings). Cheaper and cleaner than A — but it lets the Doctor decrypt ciphertexts of its own making *locally*, which removes even the query budget as a control: extraction stops being online, logged and countable and becomes free and invisible again, exactly the property §0.1 says the architecture exists to prevent. Phrase it that way rather than as "model confidentiality is gone" — the Doctor path is never fully closed (§0.1), so what Design B actually forfeits is the *visibility and countability* of it.

Design C, and why this is a trade-off space rather than an impossibility

Do not call this a trilemma. There is a third design and a reviewer will find it: the Cloud ships `D1, D2` to the *Hospital*, which solves the discrete log and returns the score sealed to the Doctor. That gives cheap decryption, result privacy against the Cloud, and — unlike Design B — keeps Doctor-side extraction online and countable. The cost is that the Hospital is back in the inference path for every query, which costs a round trip, a availability dependency, and per-query load on the KGC.

So the honest claim is narrower and still worth making: *under the constraint that the model owner is offline during inference*, you cannot have all three, and Designs A and B mark the two corners. Implement all three, tabulate which properties each holds and what each costs, and state the constraint explicitly. A characterized trade-off space with measured costs is a perfectly respectable contribution. An impossibility claim that is false is a referee report you do not want.

Note that Design B is a reasonable deployment choice if the clinic is a contractually bound licensed entity — then Doctor-side extraction is handled by rate limiting and the hash-chained audit log rather than by cryptography. Say so; do not pretend the crypto solves a governance problem.

0.6 Tiered disclosure: the extraction attack as a feature

One further move, and this is what I would build the paper's identity around. Capability separation does not only let you *deny* extraction — it lets you *grant* it selectively.

A regulator, ethics board, or independent auditor legitimately needs to interrogate the model directly. Under your architecture you can issue such a party both an encryption capability and functional-key access, which per §5.1 is precisely enough to recover the weights. The same mechanism that makes extraction an *attack* when the Cloud performs it makes it a *sanctioned audit channel* when a credentialed auditor performs it — enforced by the same capability partition, recorded in the same hash-chained audit log.

That is a coherent and unusual transparency story: the model is inspectable by the parties who ought to inspect it, and opaque to the compute infrastructure that merely evaluates it. Present it as a policy layer sitting on top of the cryptography, not as a cryptographic claim, and it holds up. It also converts your most awkward finding into a design feature, which is the kind of thing reviewers remember.

0.7 What the Cloud is actually for

Be blunt about this in the paper, because a reviewer will be blunt about it otherwise: **for one clinic and one model, the Cloud cannot be justified on compute grounds.** A 200-dimension dot product is microseconds on a laptop. Do not attempt that argument. Here are the justifications that survive scrutiny, ranked.

#	Justification	Strength / cost	Detail
0	**The model owner is offline during inference**	Strongest / free	The argument you are not making, and it is the best one you have. A two-party secure dot product via Paillier or additive secret sharing gives data privacy *and* model confidentiality, faster than FE and with well-studied security — but it requires the model owner to participate in every single query. FE does not: the Hospital issues a functional key and goes away. That is the property no MPC or additively-homomorphic approach can match, and it is what makes an untrusted third-party evaluator worth having. A reviewer *will* ask why you did not just use Paillier; this is the answer.
1	**Capability separation**	Strong / free	The Cloud is the party *structurally denied* encryption capability, which is exactly what makes it safe to hand a functional key. This is a security role, not a compute role, and it is the cleanest answer. Requires weight confidentiality; evaporates if you go fully public.
2	**Cross-institution aggregation**	Strong / cheap	The Cloud sums encrypted contributions from many hospitals: cohort prevalence, registry statistics, multi-site validation counts, "how many patients across 40 sites exceed risk threshold t." These are computations *no single party can perform*, so the Cloud is necessary rather than convenient. Cheapest way to make the third node indispensable.
3	**Multi-input fusion (MIFE)**	Strong / medium	A genomics lab encrypts RNA-seq, a clinic encrypts biomarkers, neither shares with the other, and only the Cloud can fuse them into one inner product. Your existing §9.1. Cheaper than you assumed — see the note in §10.
4	**Key-distribution broker**	Moderate / free	N hospitals × M clinics; a broker avoids pairwise key setup and gives one revocation point. Prose-only argument, no implementation needed.
5	**Governance and versioning**	Moderate / **re-key per version**	Non-cryptographic but real in healthcare: every prediction is made by a known, versioned, approved model, with no stale local copies drifting across clinics and no unauditable local modification. This is also the honest reason centralization is desirable even when secrecy is not. **But not free, and the cost is a real limitation:** functional keys are unrevocable. Once the Cloud holds `sk_y` for version 1 it can evaluate version 1 indefinitely, and the `(nonce, expiry)` pair is checked *by the Cloud* — the untrusted party — so expiry is advisory. Genuinely retiring a model version means re-running `Setup` with fresh `B` and redistributing `ek` to every clinic: a full re-key, which is precisely the N×M distribution problem row 4 says the Cloud exists to avoid. State this plainly and note revocable FE as a research direction, alongside threshold key generation. One more clause is needed and it connects to the integrity gap in §5.5: **nothing currently specifies what a clinic does when a key expires.** Expiry is Cloud-checked and the Cloud is untrusted, so there is no client-side or Hospital-side detection path at all — which is precisely why the signed anchors of §5.5 are the only mechanism that would notice a Cloud serving a retired model.
6	~~Raw compute~~	Fails	Do not use this argument. It is the one you currently imply, and it does not survive a reviewer with a calculator.

Consequence: redesign toward long-lived Cloud-held keys

Your current protocol calls `/prep_key` on every query, so the Hospital *is* online for every inference and you do not actually realize the advantage above. Under Damgård you had no choice — a long-lived key at the Cloud would be extracted at leisure. Under FHIPE, where the Cloud has no encryption capability, a long-lived `sk_y` per cancer is safe to hold indefinitely. So FHIPE does not merely strengthen a claim; it **unlocks a simpler protocol**: 33 keys issued once at provisioning, Hospital offline thereafter, no session store, no per-query round trip.

But note the tension: if the Cloud holds all 33 keys and must pick one, it learns which cancer was queried — so offline-Hospital trades directly against query privacy (§5.4a). That is a second real trade-off axis, it is cheap to measure, and it belongs in the same table as the result-privacy designs. Two orthogonal trade-offs, both quantified, is a better evaluation section than one.

The honest alternative: drop the Cloud and make this a two-party Doctor–Hospital protocol. It is cleaner, and defensible. But you would still need to explain why FE rather than local inference — and the answer would again be weight confidentiality, so you have not escaped the question, only relocated it.

Decision to make in Week 1

Route I — keep weight confidentiality, reframed (recommended). Confidentiality is justified from the deployment setting rather than from TCGA (§0.4), paired with a published inspectability package (§0.3) and tiered auditor disclosure (§0.6). Cloud justified by #1 and #2 above. Everything else in this document stands unchanged, plus the new experiment in §5.2. This is the highest-value use of the eight weeks and it does not require you to defend model IP as a value.

Route II — fully public weights, pivot the centrepiece. Then FHIPE is pointless (nothing to hide), the extraction attack is not an attack, and the paper becomes about multi-party computation the Cloud alone can do: MIFE fusion and cross-institution encrypted aggregation. Keep the quantization study, the benchmarks, the padding result, the threat model. Weeks 2–6 go to MIFE instead of FHIPE. Philosophically cleanest, higher execution risk, and less of your existing code survives. Viable — but choose it on purpose, in Week 1, not in Week 6.

The remainder of this document assumes Route I. Under Route II, sections 2 (FHIPE), 5.1–5.2 (extraction and membership inference) and 6 (XAI) change substantially; sections 3, 4, 5.3–5.5, 7 and 8 carry over almost unchanged.

1. Week-1 decisions (do not start building until these are settled)

Decision	What to do	Owner	Effort
Literature check	Before committing to novelty claims, read and cite: Kim–Lewi–Mandal–Montgomery–Raykova–Wu (FHIPE, 2018); Abdalla et al. (IPFE surveys); Dufour-Sans, Gay & Pointcheval, *Reading in the Dark*; Ligier et al. on FE-based classification; Bogos/Boureanu-style privacy-preserving genomics. Two specific jobs beyond a general survey. **(a)** Find published or deployed FE-inference systems that hand `mpk` and `sk_y` to an untrusted evaluator. If you can name two or three, your extraction result changes from "the naive design is broken" (folklore, weak) to "these published systems are vulnerable" (a finding, citable, and the thing that justifies the paper). This single task has the highest leverage on publishability of anything in Week 1. **(b)** Check whether the `g1^B` delegated-encryption technique already exists. **(c)** Before claiming no usable Python FHIPE exists, **this is now RESOLVED — do not redo it.** charm-crypto was checked and ships no inner-product or function-hiding FE, while PyMIFE does (§2.1, §16.1). The artifact claim is narrowed accordingly. Write a 1-page related-work delta.	R2 + R4	2 pd
Threat model	Write it before you harden anything — it tells you which fixes are load-bearing. Enumerate: honest-but-curious Cloud; malicious Cloud; malicious Doctor; Doctor–Cloud collusion; network adversary. For each, state which properties survive. See §5.3.	R2	3 pd
FHIPE go/no-go criteria	Write down now what "the spike succeeded" means, so you decide on evidence rather than sunk cost. **Use §8's Gate 1 wording verbatim and do not restate it here** — correctness against plaintext dot products on the `py_ecc` backend at `n` = 64, plus key and ciphertext round-tripping through JSON. No latency target and no native-library dependency, deliberately. Gate date: end of Week 3.	R1	0.5 pd
Confidentiality framing — Route I or II	The fork in §0. Decide whether weight confidentiality stays (justified as training-cohort privacy, with published inspectability and tiered auditor access) or the weights go public and the centrepiece becomes multi-party computation. Everything downstream depends on this. Do not defer it.	All	1 pd
Cloud justification	Pick from the ranked list in §0.7 — capability separation plus cross-institution aggregation is the recommended pair — and write it into the introduction. Never argue the Cloud on compute grounds.	All	0.5 pd
The dimension decision HIGHEST VALUE	Compute the **global union of active genes across all 33 cancers** from your Lasso output — five minutes of numpy. This is one decision, not two, and it must be taken together with the all-keys question of §5.4a, for a reason that is easy to miss: **all-keys evaluation is impossible without a** **common dimension.** Decryption pairs `c2` and `k2` componentwise, so a key of dimension `n_k` cannot be evaluated against a ciphertext of dimension `n_j`. Common dimension is therefore a *prerequisite* for the query-privacy mitigation, not an independent choice. See the box below for the branch structure and the exchange rate.	All	1 pd
Inspectability package	Decide what you will publish about the model: cohort provenance, feature-selection method, external and subgroup validation, calibration, failure modes, version history. This is what lets you hold the confidentiality position honestly instead of defensively.	R3	1 pd
Freeze scope	Core deliverables: FHIPE (or documented fallback), hardening, extraction attack, quantization study, benchmarks, padding, threat model. XAI and result-integrity auditing are stretch. Agree this in writing.	All	—

The dimension / all-keys decision, and its exchange rate

Adopting a common dimension `n_union` with zero weights doing model selection buys four things: one `Setup` and one `ek` instead of up to 33 (and `Setup` is `O(n^3)`); dimension fingerprinting disappears; `/model_metadata` stops leaking the Lasso feature selection, since the index set is global; and extraction lifetime scales linearly in `n`. It is also the precondition for all-keys evaluation. But the lifetime benefit must be stated in *two* units, because a single "patients per model" figure conflates two different threats:

Configuration	Patients to recover one *named* model	Patients to recover *all 33* models	Query privacy
Per-cancer `n` (~20 worst case), single key	~20	~660	Broken (dimension, then key selection)
`n_union` ~800, single key	**~800**	~26,400	Broken (key selection)
`n_union` ~800, all 33 keys	**~800**	**~800**	**Held**

So: the ~40× gain on a *targeted* model is real and is **preserved** under all-keys, because each patient still yields only one equation per model. What all-keys spends is the *suite* lifetime, which collapses from ~660 to ~800 patients for all 33 — a 1.2× change, i.e. essentially nothing. Report both numbers. If your threat is a clinic targeting one rare-cancer model, dimension buys you 40×. If your threat is wholesale loss of the model suite, all-keys gives it away and dimension does not save you.

The honest statement: under the offline-model-owner constraint, query privacy and suite lifetime are exchangeable at roughly a fixed rate of 33, and dimension is the only thing that moves the frontier outward. That belongs in §0.5's trade-off table beside the result-privacy designs — three quantified orthogonal trade-offs makes a strong evaluation section.

Branch now for the likely case: the union is too large

"Measure, do not assume" is right, but anticipate the measurement. Pan-cancer Lasso models select largely *non-overlapping* gene sets — that is what makes them discriminative — so the union is plausibly closer to the sum than to any single count: **1,000–5,000 genes**, not 800. At `n` = 2,000 on BN254, `ek` is 4×10^6 points ≈ 128 MB, `Setup` is ~8×10^9 field operations, and encryption is 4×10^6 exponentiations. §10.1 puts the FHIPE wall at `D` = 200–500, so the union is likely four to ten times past it.

- **Union ≤ ~500** → adopt it, take all four consequences, and all-keys becomes available.
- **Union ~500–1,500** → intermediate buckets, and this is the likely landing zone. Say explicitly what a partial bucket buys: the Cloud narrows the tumour type to a bucket rather than identifying it; lifetime gain is proportional to bucket dimension; `/model_metadata` still leaks *within* the bucket; and all-keys works only across models sharing a bucket, so query privacy is partial too.
- **Union > ~1,500** → unavailable under FHIPE. Then either run the dimension experiment on Damgård only, where `Setup` and encryption are linear, and report the FHIPE wall as the boundary — or drop it. Two consequences to chain here: (a) consequence (iii) holds only under FHIPE anyway, since under Damgård the weights and hence the selected features are recoverable regardless; and (b) **if the union is unaffordable then all-keys is unavailable too, so query privacy has no mitigation at all under FHIPE.** That is an honest negative result, but it must be a stated outcome rather than a surprise.

Also note what all-keys *removes* from the protocol if you do adopt it: the Doctor no longer sends a model index at all — it submits one ciphertext, receives 33 scores, keeps one. So `/model_metadata` and `model_index` leave the design entirely and the endpoint can be deleted, which is a simplification worth more than the leak mitigation it replaces.

*Role shorthand used throughout: **R1** = crypto lead (strongest mathematician), **R2** = security / threat model, **R3** = ML & data, **R4** = systems, serialization, benchmarking harness.*

2. Workstream A — Getting real function hiding

2.1 It already exists — verified. The work is different from what this section used to say.

> ***Correction, verified by running it***
>
> **PyMIFE already implements function-hiding IPE.** The module is `mife/single/fhiding/ddh.py` and its header cites eprint 2016/440 — Kim, Lewi, Mandal, Montgomery, Raykova & Wu — which is exactly the scheme written out in §2.2. It was installed and executed: correct output at `n` = 4 and `n` = 8, checked against plaintext dot products. Your original proposal's library table records "PyMIFE — FHIPE? No," and earlier revisions of this document built a 12–18 pd workstream on that entry. **The entry is wrong and the workstream was mis-scoped.** Correct the table before it reaches a reviewer.
>
> Also verified: `charm-crypto` ships *no* inner-product or function-hiding FE. Its only functional encryption is Waters' DFA scheme for regular languages — a different function class. So charm is not the counterexample §1 worried about, and the artifact claim survives in the narrowed form below.

What is actually missing — and this is your project

Gap	Evidence	What it means
Ciphertexts do not serialize	`pickle.dumps(ciphertext)` raises `PicklingError` on py_ecc's dynamically-created field classes. Functional keys (677 B) and public keys (231 B) pickle fine; ciphertexts do not. `GroupElem.export()` is declared `@abstractmethod` and the bn128 backend does not implement it — it returns `None`.	**The blocking defect.** The scheme works in one process and cannot cross a network boundary, which is the whole of your three-node architecture. This is the identical gap your proposal attributes to GoFE; it applies to PyMIFE's FHIPE as well, and that is a publishable observation about the state of distributed FE tooling.
Unusable performance on the only backend	Measured decrypt: 10.9 s at `n`=2, 17.1 s at `n`=4, 31.7 s at `n`=8 — about 3.5 s per added dimension. Projected: **3.7 min at `n`=64, 11.6 min at `n`=200, 46 min at `n`=800**, per query.	A fast pairing backend is not an optimization, it is a requirement. This also re-prices §10.1 and §1: the FHIPE dimension wall on py_ecc is a handful of dimensions, not 200–500.

The good news underneath both. PyMIFE already carries the backend abstraction §2.7 told you to build: `mife.data.pairing.PairingBase`, with `generator1`, `generator2`, `generatorT`, `pairing`, `order`. A petrelic or mcl backend implements that interface and drops in. And `FeDDH.generate(n, F=...)` already takes the backend as a parameter, so nothing needs forking.

Re-scoped Workstream A

Task	Effort	Notes
Implement `export()` / import for group elements, plus a JSON wire format for ciphertext, key and public key	3–4 pd	The core contribution. Upstream it as a PR — that is a citable artifact and it costs nothing extra.
Implement a `PairingBase` backend on petrelic or mcl	2–3 pd	Interface is five methods. Validate against the py_ecc backend at small `n`, which is now a free correctness oracle.
Delegated encryption key `g1^B` (§2.3)	2 pd	Still yours to add; PyMIFE's `encrypt` takes the full `msk`.
Integrate into the three nodes, sealed delivery, tests	2–3 pd	As before.

New total: 9–12 pd, down from 12–18. More importantly the risk profile changes: "can we implement a pairing-based scheme correctly" is replaced by "can we serialize and speed up something already known to be correct." The second is far more tractable, and the correctness oracle is free.

Consequences elsewhere — do not leave these stale

- **Gate 1 is effectively already passed.** Correctness on py_ecc exists today. Re-point the Week-3 gate at the real risk: *a round-trip of ciphertext and key through JSON between two processes, plus one working petrelic pairing.*
- **The artifact claim narrows and sharpens.** Not "first usable Python FHIPE" — that is now false and would be caught. Claim instead: *serialization and a performant backend for an existing implementation that could not previously be used across a network.* That is the real gap, and it is the one your own proposal identified.
- **§10.3 gets much cheaper.** `mife/single/quadratic/ddh.py` exists and imports cleanly, so degree-2 FE needs no Go and no new scheme.
- **The aggregation story gets cheaper.** `mife/multi/damgard.py`, `mife/multiclient/damgard.py`, `mife/multiclient/rom/ddh.py` and `mife/multiclient/decentralized/{ddh,palia}.py` all import. §0.7 rows 2 and 3 are within reach rather than future work.
- **Post-quantum moves from discussion to option.** `mife/single/lwe.py` exists.
- **Capacity improves by 3–6 pd.** Re-derive the §8 total.

2.2 The scheme, written out

Retained for the paper and for understanding what you are serializing — you no longer need to implement it from this description. Read it alongside `mife/single/fhiding/ddh.py`, which follows it closely.

Type-3 bilinear group: `e : G1 × G2 → GT`, prime order `p`, generators `g1, g2`. Vector dimension `n`.

```
Setup(n):
    B  <-$ GL_n(Z_p)              # random invertible n x n matrix
    d  =  det(B)
    B* =  d * (B^-1)^T
    msk = (B, B*, d)              # Hospital only

Encrypt(msk, x):                  # x in Z_p^n
    a  <-$ Z_p*
    c1 = g1^a
    c2 = g1^(a * (x . B))         # n elements of G1
    return (c1, c2)

KeyGen(msk, y):                   # y = quantized weights
    b  <-$ Z_p*
    k1 = g2^(b * d)
    k2 = g2^(b * (y . B*))        # n elements of G2
    return (k1, k2)

Decrypt(ct, sk, bound L):
    D1 = e(c1, k1)                                  # = gT^(a*b*d)
    D2 = PRODUCT over i of e(c2[i], k2[i])          # = gT^(a*b*d*<x,y>)
    return z in [-L, L] such that D1^z == D2        # discrete log, base D1
```

Correctness: $(xB)\cdot(yB^*) = x\,B\,B^{*T}\,y^T = x\,B\,(d\cdot B^{-1})\,y^T = d\cdot\langle x,y\rangle$, since $B^{*T} = d\cdot B^{-1}$. The masks `a` and `b` cancel in the exponent ratio, which is precisely why `sk_y` leaks nothing about `y` beyond the inner product — each key is a fresh random re-basing of `y`.

2.3 The delegated encryption key — your design contribution

The naive deployment gives the Doctor `msk` so it can encrypt. Do not do this. From `B` an attacker computes B^{-1}, hence B^*, hence can mint functional keys for arbitrary `y` — a compromised clinic terminal becomes a key-generation centre and can decrypt other clinics' traffic. Instead, ship the Doctor an **encryption key in the exponent**:

```
ek = { g1^(B[i][j]) }  for all i, j        # n^2 elements of G1

Doctor encrypts without ever learning B:
    t[j] = PRODUCT over i of ( g1^B[i][j] )^x[i]     # = g1^((x.B)[j])
    c2[j] = t[j]^a ,  c1 = g1^a
```

Recovering `B` from `ek` is n^2 discrete logs in G1 — infeasible. So a compromised Doctor can encrypt but cannot mint keys, cannot decrypt, and cannot recover the master secret. This is a small but real construction result, and it is exactly the kind of "useful and new" that an undergraduate paper can legitimately claim. Report it with its cost, which is the honest part: encryption becomes $O(n^2)$ group exponentiations instead of $O(n)$ field multiplications.

> ***Do not claim proven FHIPE security for the modified scheme***
>
> The KLMMRW function-hiding proof is in the **secret-key** setting, where the adversary has no encryption capability at all. Handing out `ek` changes the security game, so your construction is *not covered by the original theorem*. Recovering `B` from `g1^B` is discrete-log hard, but that is an argument about one attack, not a proof that function hiding survives.
>
> A crypto reviewer will catch this immediately, so get there first. Say plainly: the modification is not covered by the KLMMRW proof; we give an informal argument for why it resists weight recovery; a formal treatment in the secret-key-with-delegated-encryption model is future work. Stated that way it is honest and entirely acceptable for this scope. Claimed as "FHIPE security" it is the single most likely reason this paper gets rejected. Also check hard whether the `g1^B` technique is already folklore — encryption keys in the exponent is a standard move and it may well appear in prior work; if so, cite it and drop the novelty claim rather than having it found for you.

Mitigations for the `n²` cost, worth measuring: (i) multi-exponentiation — Pippenger or a windowed method computes the product-of-powers far faster than `n` separate exponentiations; (ii) sparsity — measure how many quantized `x_i` are actually zero before relying on this, and note that the Lasso sparsity you are used to lives in `y`, which never enters `ek` at all; expect a modest constant-factor gain, not asymptotic relief; (iii) the `t[j]` vector depends only on the patient, so for repeated queries about the same patient (the XAI case in §6) it is computed once and reused across all queries. Point (iii) matters: it means the per-gene XAI queries are far cheaper than the first one.

2.4 Two findings you will hit, and should publish

Finding 1: FHIPE decryption cannot amortize its discrete-log table

In Damgård IPFE the baby-step giant-step table is built over a *fixed* base and can be precomputed once at boot — which is what your current `rainbow_table` stub was reaching for. In FHIPE the base is `D1 = gT^(a·b·d)`, which is **fresh for every ciphertext–key pair**. No table is reusable. Recommendation: use **Pollard's kangaroo** (lambda) method instead of BSGS — it is also `O(sqrt(L))` but needs almost no memory and no table build. This is a concrete, quantifiable cost of function hiding that as far as I know nobody has written up for a deployed FE pipeline. Measure it and put it in the paper.

Finding 2: your search-space profiling stops being optional

Because the discrete log now costs `O(sqrt(L))` GT operations *per query* with no amortization, your current `L = 5,000,000` is directly on the critical path. Profiling the true inner-product range across all 33 cancers and all TCGA patients (already listed in your Section 8) becomes a load-bearing optimization rather than a nice-to-have. Expect a large speedup and a clean figure out of it.

2.5 Routes, ranked by risk-adjusted reward

#	Route	Effort	Reward	Risk
1	**Extend PyMIFE's existing FHIPE** — serialization plus a fast backend. RECOMMENDED	9–12 pd (R1, R4 assisting from W4)	Highest, and lower risk than when this table was written: the scheme exists and is verified correct (§2.1), so you are repairing rather than building. Wire format is yours to define; upstreaming it is a citable artifact. Whole stack stays Python.	Low–medium. Concentrated entirely in the pairing-library install and GT performance. The py_ecc backend is a free correctness oracle throughout.
2	**Fork GoFE, add `MarshalBinary`** to the FHIPE key/ciphertext types in-package; run a Go crypto sidecar per node, Python nodes talk to it over local HTTP.	10–15 pd	Good. Battle-tested crypto, fast. Upstream PR is a nice artifact. Unexported-field problem is solvable from *inside* the package — the underlying types already marshal.	Medium-low technically, but polyglot: 3 sidecars, IPC, deployment complexity. Your ML stays Python regardless, so you pay FFI cost anyway.
3	**Rust core (`blstrs` / arkworks) + PyO3 bindings.**	15–22 pd	Best performance and cleanest serialization. BLS12-381 at full 128-bit security.	High for your timeline unless someone already knows Rust. You are implementing FHIPE either way, so the only gain over Route 1 is speed.
4	**Stay on PyMIFE.** Keep Damgård IPFE and rely on operational defences alone. SCHEME FALLBACK	2–3 pd beyond the mandatory work	Moderate. Note carefully: the query-budget and noise work is **not** what distinguishes this route — that is required in every route (§0.1), because it is the only defence against the Doctor-side path. What this route gives up is protection against the *Cloud*, which under Damgård extracts offline and for free.	Low execution risk, materially weaker result. Adopt only if Gate 1 fails.

Recommendation: commit to Route 1, now clearly correct given §2.1. Routes 2 and 3 exist to work around a scheme you no longer need to write, so treat them as historical unless the petrelic backend proves impossible. If the gate fails, fall to Route 4 and spend the reclaimed time on the empirical workstreams — which are strong enough to carry a paper by themselves. Route 2 only if R1 is more comfortable in Go than in bilinear-group Python.

2.6 Pairing library options

Library	Curve / security	Speed	Serialization	Verdict
`petrelic` (RELIC bindings)	BN254 — note this is roughly **100-bit**, not 128, after Kim–Barbulescu	Fast (pairing on the order of milliseconds)	Yes — `to_binary()` / `from_binary()` on all groups incl. GT	**Primary performance backend.** Clean multiplicative API that maps directly onto the pseudocode. Install may need build tooling; test this on day 1. Disclose the curve's security level honestly in the paper — do not claim 128-bit.
`py_ecc`	BLS12-381 — genuine 128-bit	Very slow (pure Python; pairings in the hundreds of ms)	Trivial — plain Python integers throughout	**Correctness reference backend.** Pure Python, pip-installs anywhere, no build step. Use it to validate the scheme at n = 8–32 in days rather than weeks. Never benchmark on it.
`mcl` bindings	BLS12-381 / BN254	Fastest of the Python options	Yes, native	Good alternative to petrelic if that install fights you. Binding maturity varies by package; budget a day to evaluate.
`charm-crypto`	MNT / BN curves	Moderate	Yes, mature	Works, and has the best FE-research pedigree, but installation (PBC, GMP, older Python assumptions) is historically the most painful of the set. Last resort.
`blspy` / `blst`	BLS12-381	Very fast	Yes for G1/G2	Built for BLS *signatures*; GT is often not fully exposed, which you need. Check before committing.

2.7 The two-backend plan — already half-built

PyMIFE ships `mife.data.pairing.PairingBase` with exactly the seam described below, and `FeDDH.generate(n, F=...)` accepts a backend. So do not write the interface — implement a second class against theirs. The py_ecc backend becomes your reference oracle for free.

This is the single most important tactical decision in the project. Define a small interface and implement the scheme *once* against it:

```
class PairingBackend:
    p                       # group order
    def g1(), g2()          # generators
    def g1_exp(P, k), g2_exp(Q, k)
    def gt_mul(A, B), gt_exp(A, k), gt_eq(A, B)
    def pair(P, Q)
    def pair_product(Ps, Qs)    # product of pairings; fall back to a loop
    def ser_g1(P) / deser_g1(b) # and G2, GT
```

Then: **Week 2** implement `PyEccBackend` and get the scheme provably correct at small `n` against a plaintext dot product. **Week 3** implement `PetrelicBackend` and re-run the identical test suite. Correctness and performance become independent problems, and neither can block the other. It also gives you a legitimate reproducibility story: the artifact runs anywhere on `py_ecc`, fast on `petrelic`.

Optimization to look for: a *product-of-pairings* primitive. Computing `PRODUCT e(c2[i], k2[i])` as one operation shares the expensive final exponentiation across all `n` Miller loops, typically a 2–3× speedup on the dominant cost. If your binding exposes it (RELIC has simultaneous pairing internally), use it; if not, the fallback loop is correct but slower — and the delta is a nice line in your latency table.

2.8 Architecture changes that FHIPE lets you make

Two changes that stay entirely within your existing three-node proposal but remove attack surface structurally rather than by adding checks:

Change 1: sealed key delivery — delete `/fetch_key` entirely

Today the Hospital stores `sk_y` in `active_sessions` and the Cloud pulls it. That endpoint is unauthenticated, so the Doctor can pull its own `sk_y` and extract locally — the worst path in your current system. Instead: the Hospital **encrypts `sk_y` under the Cloud's long-term public key** (X25519 sealed box) and returns the opaque blob to the Doctor, which relays it with the ciphertext. The Doctor cannot open it. The Cloud can.

This deletes an endpoint, deletes the session store, and therefore deletes your no-TTL leak and your replay-store logic in one move. **Decided: keys are long-lived and the Hospital is offline.** An earlier draft bound each sealed key to a specific ciphertext hash, which is tighter but forces the Hospital to act on every query — destroying the offline-model-owner property that is your single strongest justification (§0.7, row 0). That binding was defending against a threat FHIPE already removes: a Cloud with no encryption capability cannot do anything harmful with a key, however long it holds it. So drop the ciphertext binding. Provision 33 sealed keys once at setup, one per cancer, and the Hospital is never needed again during inference. Replay protection reduces to a `(nonce, expiry)` pair the Cloud tracks, which is all that is required once keys are not secret-per-query.

Change 2: `ek` is clinic-only, and that boundary is the security theorem

Serve the delegated encryption key from a separate endpoint gated on mTLS client-certificate identity, issued only to clinic certs. The Cloud has no client cert for it and therefore no encryption capability. The claim this supports is narrower than it first appears: *no single node can extract the model offline and undetectably; the Cloud cannot extract at all, and the residual path through the Doctor is online, authenticated, logged and rate-limitable.* Checkable, and honest. Do **not** write "extraction requires Doctor–Cloud collusion" — it does not (§0.1).

3. Workstream B — Security hardening

Do the scheme-agnostic items in Weeks 1–2; they are independent of the FHIPE outcome. The serialization work folds into the FHIPE implementation rather than being done twice. The right-hand column tells you what changes if you land FHIPE.

Issue	Fix	Tool	Effort	Under FHIPE
Shared `NETWORK_SECRET` gives all three nodes, including the untrusted one, the same signing key — no origin authentication, and the Cloud can forge Doctor traffic	Per-node Ed25519 keypairs. Each node holds its own private key and the others' public keys. Sign the canonical payload bytes; verify sender identity, not just integrity.	`PyNaCl` or `cryptography`	1.5 pd	Same, and now load-bearing: it is what stops the Cloud from impersonating a clinic to obtain `ek`.
pickle RCE — HMAC authenticates the payload but the secret is hardcoded in a file the untrusted Cloud runs, so the	Remove pickle. Explicit field-wise serialization: every object type gets a schema, curve points go out as compressed bytes, base64 in JSON. Reject unknown fields.	stdlib `json`; `pydantic` for schemas if you like	Folded into FHIPE (2 pd standalone)	Free — you are defining the wire format anyway. This is the strongest argument for Route 1.

precondition for forgery is already met				
Doctor generates session IDs; KGC should own its own namespace	`/prep_key` returns a Hospital-generated ID.	stdlib	0.5 pd	Dissolves — sealed delivery removes session IDs from the protocol.
`/fetch_key` **unauthenticated** — malicious Doctor extraction path	Sealed key delivery (§2.8, Change 1). Endpoint deleted.	`PyNaCl` SealedBox	1.5 pd	Becomes the core of the security argument.
`verify=False` **everywhere**, adhoc TLS certs	Local CA, per-node server certs, mutual TLS with client-cert verification. Client-cert subject becomes the node identity used for authorization.	`openssl` / `step-ca`, Flask `ssl_context`	1 pd	Same, plus it is the gate on `ek` distribution.
`hospital_vault` **keyed by** `vector_len` — two cancers with equal active-gene counts silently share a master key, so a key for cancer A decrypts a cancer-B ciphertext into a plausible wrong number	Key by `cancer_index` alone plus a domain-separation tag. Do *not* key by `(cancer_index, n)` — once you pad every vector to `n_max` (§5.4a), `n` is constant and the collision returns. Never overwrite an existing entry. Add a domain-separation tag inside the ciphertext so cross-cancer use fails loudly instead of silently.	—	1 pd	Same bug class with `B` instead of `msk`. Fix it in the new code from the start.
`KeyError` **if** `/prep_key` **precedes** `/public_key` for a given dimension	Lazy-initialize inside `prep_key`, or make setup explicit and ordered in the protocol.	—	0.25 pd	Same.
No input validation — unbounded `model_index`, and a large `vector_len` triggers key generation that can exhaust memory (a trivial DoS)	Bounds-check `model_index` in [0,32]; whitelist `vector_len` against dimensions the model actually uses; reject everything else.	`pydantic`	0.5 pd	Same, and more important — `Setup(n)` is `O(n^2)` here.
No rate limiting	Counters enforced **at the Cloud**, since it is the only node that sees every request and the Hospital is offline by decision. Granularity is **per** `(model, key-evaluation)`, not per request and not per clinic — see the box below for why both of those are wrong. Per-clinic certificates give you sub-allocations beneath the global cap, and the Hospital audits the Cloud's signed counters periodically. This also bounds the audit rate for §5.5.	`Flask-Limiter` at the Cloud + signed counters	0.5 pd	Same.
Dead code — `safe_serialize()` in doctor_client, `safe_deserialize()` and the empty `rainbow_table` in cloud_node, unused `uuid` / `base64` / `json` imports in hospital_node	Delete. Note in the paper that the `rainbow_table` idea is *impossible* under FHIPE (Finding 1, §2.4) — that turns leftover scaffolding into an insight.	—	0.25 pd	—
Unspecified behaviour when the true inner product falls outside the search range — a silent wrong answer in a diagnostic pipeline	The discrete-log search must fail *loudly*, not return a plausible neighbouring value. Return an explicit out-of-range error to the Doctor and log it. Set the range from your Week-2 profiling with margin, and treat any out-of-range event as a bug to investigate rather than an expected outcome.	—	0.5 pd	Interacts with perturbation and Design A, both of which widen the range and both of which cost `O(sqrt(L))` in decryption time (§2.4, Finding 2). Budget the three together, not separately.
No structured logging or audit trail — a clinical system needs one, and reviewers of healthcare papers look for it	Append-only audit log at the Hospital: who requested what key, when, under which certificate. Hash-chain the entries so tampering is detectable.	`structlog`	1 pd	Same. Also the data source for the extraction-detection discussion.
No query budget — the primary defence against the inherent Doctor-side extraction path (§0.1)	**Global** per-model query accounting enforced at the Cloud, with per-clinic allocations beneath a global cap. See the box below on why per-clinic alone fails and why the counter cannot live at the	`Flask-Limiter` + signed counters	2 pd	Unchanged and still required. FHIPE does not remove this need.

	Hospital.			
No input bounds on the query vector — without them, output perturbation is defeated outright	Reject queries whose quantized vector exceeds a plausible norm for real patient data. Derive the bound from the TCGA distribution.	NumPy	0.5 pd	Prerequisite for the perturbation work below, not optional.
No output perturbation	Calibrated discrete noise on the returned score, applied by the Cloud in the exponent (see box). **Truncate the distribution at a stated quantile** — discrete Laplace has unbounded support, so untruncated noise pushes results outside `[-L, L]` at a nonzero rate and turns the loud-failure rule below into a false alarm. Widen `L` by exactly the truncation bound, compute the residual out-of-range probability, and make the error distinguish "noise tail, expected at rate p" from "outside profiled range, investigate." Report the accuracy curve as a *measured characterization*, never as a formal guarantee.	NumPy	2 pd	Interacts with §5.5; see the tolerance note there.

How the perturbation actually has to work — three things that are easy to get wrong

Who adds the noise. Not the Doctor (it is the adversary), not the Hospital (noise baked into a key is fixed, so averaging removes it), and the Cloud cannot add it to a value it is not supposed to read. The way out: the Cloud already holds `D1 = g_T^(abd)` and `D2 = g_T^(abd·<x,y>)`, so for any integer `e` it computes `D2' = D2 · D1^e`, which decrypts to `<x,y> + e`. *The Cloud applies fresh per-query noise to a value it never learns.* Combine with Design B and you get result privacy against the Cloud and perturbation against the Doctor at once, at no cryptographic cost. Use a discrete mechanism (geometric or discrete Laplace) since `e` must be an integer — which your quantized domain already gives you. Caveat to state: an untrusted Cloud could set `e = 0` and nobody would know, so this is conditional on an honest-but-curious Cloud.

Two attacks that make noise useless on its own. *Averaging* — ask the same question `m` times and the noise falls as `1/sqrt(m)`, so noise means nothing without a query budget; the two are not independent defences and the budget is doing most of the work. *Rescaling* — submit `c·e_i` instead of `e_i`, receive `c·y_i + e`, divide by `c`; the signal scales and the noise does not. This is why the input-norm bound above is a prerequisite rather than hygiene.

A third attack the norm bound cannot see, and why there is no cheap fix. Nothing in the scheme binds `c1` to `c2`. A Doctor can send `c1 = g1^a` alongside `c2[j] = t[j]^(c·a)` using a completely ordinary, in-norm patient vector. Then `D2 = D1^(c·<x,y>)`, the Cloud adds noise against the `D1` base as specified, and decryption yields `c·<x,y> + e`. Divide by `c`: the signal scaled, the noise did not. The plaintext passes every check because *the attack lives in the randomness, not the input*. There is no cheap defence — verifying that `c1` and all `c2[j]` share one exponent needs `t[j]`, and the Cloud only ever sees `t[j]^a`. Moving the noise into the key makes it scale with `c`, but then it is fixed per key and averaging removes it; fresh noise per key puts the Hospital back online. The constraint is circular.

How bad is it, actually. Less bad than it first appears, because of the loud out-of-range rule (§3): if `c·<x,y>` leaves the profiled range `[-L, L]` the query fails visibly. The exact constraint is `c ≤ L / |<x,y>|` — `L` over the score of *the query the adversary chose to submit*, not over the maximum score magnitude. That distinction matters, because the adversary picks `x`. Given a partially recovered `y_hat` with relative error `eps`, it chooses `x` in the near-null space of `y_hat` — an `(n-1)`-dimensional subspace, so plausible in-norm vectors are abundant — making `|<x,y>| ~ eps·S` and therefore `c ~ (L/S)/eps`. The noise is suppressed by that same factor, so the error shrinks by roughly `eps` per round: **geometric convergence.** A naive bound of 2–10× is the *first-round* figure only, and quoting it in the paper invites a reviewer to find the null-space trick and conclude you did not. **Do not try to fix this properly.** Report it as a negative result about delegated-encryption FE, and report it at full strength: ciphertext well-formedness is unverifiable by the evaluator in this construction, so Cloud-side perturbation is *fully defeated* by an adaptive adversary submitting ordinary-looking vectors — not merely weakened. Tight range profiling still helps against a non-adaptive one, which is worth a sentence, but do not present it as the bound. ~0.5 pd to write.

Why it helps at all, and the argument to make. Diagnosis reads a sum over `n` terms; extraction reads a single coefficient. The same absolute noise is therefore proportionally far more damaging to extraction than to the clinical score, and that asymmetry is measurable. Use it as the justification. It does not require a differential-privacy claim, and it does not route through §5.2 — which matters, because §5.2 is optional and may return nothing.

> ***Per-clinic budgets do not bound a global quantity***
> The model is identical for every clinic, so extraction effort *pools*. Two colluding clinics, one clinic with two certificates, or a clinic that re-enrols each get `n` queries against an `n`-query model. Per-identity limits are the wrong granularity for the thing you are protecting.
>
> **Granularity, precisely.** Under all-keys evaluation (§5.4a) a single request consumes one unit of lifetime from *each* of 33 models. So a counter that counts *requests* under-counts leakage by 33×, and one that counts *key evaluations* against a single shared budget exhausts 33× too fast. The counter must be **per** `(model, key evaluated)`, incremented once per key per request. Both the row above and the original phrasing of this box predate the all-keys decision and neither said this.
>
> Enforce it **at the Cloud**, with per-clinic budgets as sub-allocations. It cannot live at the Hospital — the Hospital is offline by decision, and putting a per-query counter there would undo that. Have the Hospital audit the Cloud's signed counters periodically instead. Add a "colluding clinics" row to the coalition table, and state plainly that clinic enrolment is now a security-relevant gate: a governance control, not a cryptographic one, exactly as with the auditor channel. ~0.5 pd, and it belongs in Week 2 beside the budget itself.

Total Workstream B: ~14–15 pd, mostly R2, front-loaded into Weeks 1–3. *R2 is the bottleneck role at this load — move the literature delta to R4, who is light in Week 1 and needs the related work for the artifact claim anyway.*

4. Workstream C — Correctness of the ML pipeline

These are bugs, not enhancements, and at least one of them is probably degrading your headline accuracy right now. Do them in Week 1 — every downstream number depends on them.

4.1 Two bugs to fix immediately

> **Truncation, not rounding.** `(scaled * 100).astype(int)` truncates toward zero. Lasso coefficients on standardized RNA-seq are routinely in the 0.001–0.05 band, so `0.004 * 100 = 0.4 → 0` — you are silently deleting genes that `/model_metadata` reports as active, and distorting the ratios among survivors. Use `np.rint(...).astype(np.int64)`. Then stop using one factor for both vectors: choose `Q_w` from the coefficient distribution and `Q_x` from the feature distribution independently, and divide the result by `Q_w * Q_x`. Report the chosen values with the distributions that justified them.
>
> **Missing-gene imputation injects fake signal.** `reindex(columns=..., fill_value=0)` fills absent genes with a raw zero, which the scaler then maps to `(0 - mu)/sigma`. For log-scale RNA-seq with mu around 5–10 that is an extreme negative z-score, not a neutral value. Impute with `scaler.mean_` so missing genes contribute zero after scaling, and log how many genes were imputed per patient — if it is a large fraction, that is itself a finding worth a sentence.

4.2 Experiments

Experiment	Method	Tool	Effort
Quantization vs. accuracy	Sweep `Q_w` and `Q_x` over a grid; report macro-F1 and per-cancer F1 against the unquantized float baseline across all 33 classes. Include the truncation-vs-rounding comparison as its own row — it quantifies the bug you just fixed and makes the point that naive integer casting is a correctness hazard in FE pipelines generally.	scikit-learn	3 pd (R3)
Inner-product range profiling	Compute true `<x,y>` for every patient × every cancer in plaintext. Report min/max/percentiles per cancer. Set `L` per-cancer from the empirical bound plus margin. Now on the critical path (§2.4, Finding 2).	NumPy	1.5 pd (R3)
Probability calibration	An angle you have not considered: quantization perturbs the logit, and the sigmoid is steepest near 0.5, so *absolute error in the reported risk percentage is largest exactly where clinical decisions are marginal*. Plot reported vs. true probability across the range and show reliability curves. This is a clinically meaningful result that is cheap to produce and unusual in FE papers.	sklearn calibration	1.5 pd (R3)
Baseline comparison	Same task, four systems: plaintext; CKKS via TenSEAL; your Damgård pipeline; your FHIPE pipeline. Report latency per phase, ciphertext and key sizes, accuracy, and what each protects. Not novel, but its absence is the first thing a reviewer complains about.	TenSEAL	3 pd (R4)
Latency breakdown	Instrument all five phases. For FHIPE, break out `ek`-based encryption, product-of-pairings, and discrete-log search separately — the split is the interesting part. Sweep `n` to get scaling curves.	`pytest-benchmark`	2 pd (R4)

5. Workstream D — The cheap research additions

Every item here is small, and every one produces a figure or a table. Ranked by reward per person-day.

5.1 The extraction attack, measured HIGHEST PRIORITY

This is what motivates your entire paper. Without it, the FHIPE upgrade reads as gold-plating; with it, the upgrade is a response to a demonstrated break. Do it in Week 1–2, on your *current* Damgård system, before you change anything.

```
for i in range(k):
    ct = FeDamgard.encrypt(unit_vector(i, k), mpk)
    y_hat[i] = FeDamgard.decrypt(ct, mpk, sk_y, bound)   # == y[i], exactly
```

Note that you recover the *quantized* model, not the original float model — say so, or the surrogate-fidelity figure is ambiguous. Deliverables: queries and wall-clock seconds to recover each of the 33 quantized weight vectors from a single `sk_y`; surrogate *fidelity* (agreement with the real model's predictions, not just exactness) and completeness as a function of query budget, so attack and defence share one axis; recovery accuracy (it should be exact, which is the point); a plot against active-gene count; and the downstream consequence — train a surrogate model on the recovered weights and show it reproduces the hospital's predictions. Then demonstrate the same attack fails under FHIPE, and that it succeeds again under simulated Doctor–Cloud collusion. That triple is your central figure.

Effort: 2 pd (R2). Highest reward per unit effort in the entire project.

5.2 From weight recovery to training-cohort leakage OPTIONAL — see §0.4

Compose the two attacks. Stage 1 recovers the weights (§5.1). Stage 2 asks whether holding the coefficients tells you anything about the cohort that produced them. Per §0.4 this is a *methodological measurement on a public surrogate* — it bounds what leakage would look like on a private cohort of comparable dimensionality. It is not a claim of harm to TCGA participants, and you should say so in the same breath as the result.

Method. Hold out a set of TCGA samples at training time. Recover the weight vectors via §5.1. Then attempt to distinguish training members from non-members using only the recovered weights — per-sample loss or confidence thresholding in the style of Yeom et al. is the standard cheap baseline, and a Fredrikson-style model-inversion attempt on a single target gene given partial covariates is the stronger version. Report AUC, and stratify by cancer type: the rarer classes (CHOL, UVM, DLBC each have on the order of tens of samples in TCGA) have far worse sample-to-parameter ratios and should leak more, which is a clean and testable prediction.

Be prepared for either outcome, and say so in the paper. Strong Lasso regularization may destroy most of the signal, and a null result is genuinely publishable: "*weight recovery does not measurably compromise training-cohort*

membership at this scale and regularization strength" is a useful calibration of a threat that is widely asserted and rarely measured. But if AUC comes out materially above 0.5 — and the rare-cancer stratification is where I would expect it to — you have a supporting argument that weight confidentiality matters in the private-cohort deployment, which strengthens §0.4 without carrying the paper on its own.

Effort: 3 pd (R2 with R3), Week 6, and genuinely optional. Tools: scikit-learn plus a hand-rolled threshold attack — forty lines, do not over-engineer it. Cut this before anything in §0.5 or §5.4. It is a nice supporting measurement, not a spine.

5.3 Threat model and coalition table

No code, two pages, and it is the largest single gap between what you have and a publishable paper. Enumerate adversaries and state per-coalition what survives. Sketch:

Adversary	Patient privacy	Model confidentiality	Notes
Honest-but-curious Cloud	Holds	**Holds only under FHIPE**	Under Damgård, broken by §5.1 with no deviation from protocol — the Cloud need not even misbehave.
Malicious Cloud	Holds	Holds under FHIPE	*Result integrity does not hold* in either scheme — see §5.5.
Malicious Doctor	N/A (owns the data)	**Broken inherently**	Not merely unpreventable but **automatic**: `n` legitimate patient queries suffice, with no chosen inputs and no deviation from protocol (§0.1). Sealed delivery and FHIPE remove the `/fetch_key` shortcut but not the ordinary clinical path. Bounded in lifetime by dimension, in rate by the budget, and made attributable by the log — not prevented by any of them.
Doctor & Cloud colluding	N/A	**Broken**	Unavoidable: the coalition holds an encryption capability plus a functional key. State this as an explicit assumption rather than letting a reviewer find it.
Colluding or Sybil clinics	N/A	**Broken**	Extraction pools across identities against a single shared model. Bounded only by a global per-model query cap; per-clinic limits do not help (§3).
Malicious or compromised Hospital	**Fails**	N/A (owns it)	Currently absent and a reviewer will ask. The Hospital holds `msk`, `B`, every functional key and the audit log, so it is a single point of total failure. One clause makes this both more accurate and less alarming: under sealed-key delivery the Hospital never sees ciphertexts, so patient privacy fails only under Hospital+Cloud collusion or Hospital+network access, not from the Hospital alone. State it as an explicit assumption and note threshold key generation or a split KGC as future work.
Unavailable or selectively-failing Cloud	Holds	Holds	Absent from the model. A Cloud that returns errors, or answers some queries and not others, is neither lying nor detectable by the linearity audit of §5.5. In a diagnostic pipeline that is a *safety* property, not just a liveness one — a silently dropped result is a test that did not happen. Needs a stated expectation and a client-side timeout-and-escalate path.
Network adversary	Holds (TLS + signatures)	Holds	Query-pattern leakage survives TLS — see §5.4.

Effort: 3 pd (R2), Week 1. Do it first; it tells you which hardening items matter.

5.4 What the Cloud learns at inference time PROMOTED TO CORE

Two leaks, one section, and together they are the strongest inference-side contribution available to you (§0.5). Do both; they share a measurement harness.

(a) Query privacy — which tumour type is being tested

In the original design the Cloud saw the ciphertext dimension, and Lasso active-gene counts differ per cancer, so `n` was

a near-unique fingerprint for the tumour type queried. That is the clinically sensitive fact: it reveals which primary site or tumour type is under investigation for a named patient — precisely what is at stake in a tumour-of-unknown-primary workup — and is arguably more sensitive than the expression values themselves.

> **Padding is no longer the mitigation here.** Under the long-lived-key decision (§2.8) the Cloud holds 33 keys and *selects* one per request, so it learns the tumour type from key selection regardless of dimension. Padding now addresses a channel the new architecture already closed, while the live channel goes unmitigated — a downstream section that was not revisited when the key-lifetime decision changed. The mitigation that works is to **evaluate against all 33 keys and return all 33 results**, the Doctor discarding 32. Cost is 33× the pairing work: at `n` = 200 that is 6,600 pairings, roughly 7–13 s before a product-of-pairings optimization. *The trade is not free:* the Doctor now receives 33 scores per patient instead of one, so under the lifetime result (§0.1) every model leaks in the time it currently takes one to leak. You are buying query privacy with model lifetime. Measure both and state the exchange rate.

Quantify it first (how many of the 33 cancers are uniquely identified by `n`?), then pad all vectors to a common dimension `n_max` with zero-weight slots, and measure the latency and ciphertext-size cost. Under FHIPE the padding cost is higher because encryption is `O(n^2)`, which makes the privacy/performance trade-off a real curve rather than a footnote. Consider intermediate padding buckets as a middle point.

(b) Result privacy — the risk score itself

Implement all three designs from §0.5 and measure the trade-off space. Concretely: (i) baseline, Cloud decrypts and learns the score; (ii) Design A, Hospital-blinded score, sweeping blinding width `k` — note that `k` is **not freely choosable**: the search cost is `2^((k+20)/2)` group operations against a score range of roughly `2^20`, so a one-to-ten-second budget caps `k` at about 20 bits *in isolation*. The deployable ceiling is lower: perturbation widens the same range (§3) and all-keys multiplies the pairing cost by 33, so take the ceiling from §12's joint blinding-plus-perturbation row rather than from the blinding-only figure. Present it as a bounded range with a hard ceiling, not a smooth open-ended curve — and plotting Cloud-side decryption latency against statistical leakage `S/R`; (iii) Design B, split decryption, reporting the GT payload size, the Doctor-side search time, and the extraction capability it concedes. Present the three as a table of which properties each holds, and (iv) Design C, Hospital-in-the-loop, reporting the added round-trip latency and KGC load. Tabulate which of the three properties each design holds and what it costs. Frame the finding as a trade-off space under the offline-model-owner constraint, not as an impossibility (§0.5).

Also worth one paragraph: your `clinical_intercepts.json` is described in the source as "harmless statistical intercepts." Given the raw inner product, the intercept is exactly the missing ingredient that converts a monotone score into a calibrated probability. It is not harmless; it is the last mile. Keeping it Doctor-side is the right call, but say *why* rather than calling it harmless.

> **All-keys evaluation is more expensive than the line below suggests, and it breaks the estimates in (b).** The "2 pd" figure was written when (a) meant padding. Implementing 33-key evaluation — key provisioning and storage, the evaluation loop, the discard logic, and the measurement — is **4–5 pd**, and it multiplies everything in (b): Design A needs 33 blinded keys with 33 distinct `rho` values delivered to the Doctor; Design B has the Doctor solving 33 discrete logs per query on clinic hardware, over a range already widened by blinding and the noise tail; Design C ships 33 GT pairs to the Hospital. Re-price (b) at **6–7 pd** under all-keys and say in the trade-off table which key count each design assumes, or the two halves of this section describe different systems. It also needs a schedule slot — currently it has none.

Effort: 4–5 pd for (a), 6–7 pd for (b) under all-keys (2+4 if you stay single-key), R4 with R1 on the blinded key construction. Weeks 4–6. This is the highest-value new work in the revision, and the largest single addition to the capacity total in §8.

5.5 Probabilistic result-integrity auditing

Nobody in your system checks the Cloud's arithmetic. It can return any integer and a clinician acts on it — that is a patient-safety hole, not merely a security one, and FE-inference papers largely ignore it. Full verifiable FE is a research project; a probabilistic audit is a week.

Mechanism — use linearity, not pre-shared answers. The obvious design has the Hospital supply decoy vectors with known answers, but that puts the Hospital back in the inference path and quietly destroys the offline-model-owner property that is your main reason for using FE at all (§0.7, row 0). Pre-provisioning a batch at setup works, but there is a better construction that needs the Hospital not at all.

The inner product is linear, so the Doctor can audit against its own history. Take two previously answered queries `x1, x2` with returned scores `s1, s2`, pick random coefficients `a, b`, and submit a fresh encryption of `a·x1 + b·x2`. The correct answer is `a·s1 + b·s2`, which the Doctor already knows. The Cloud cannot distinguish the audit query from a genuine one — it is a valid ciphertext of a valid input vector, freshly randomized. A wrong answer proves misbehaviour, and the Hospital was never involved.

> **This audit and output perturbation cancel each other unless you handle it.** The check is that the reply equals `a·s1 + b·s2`, and noise breaks exactly that linearity — a noisy Cloud and a cheating Cloud produce the same observable. Fix: accept replies within `±tau` of the linear prediction, with `tau` derived from the noise scale — noting that the check spans *three* noisy values (`x1`, `x2` and the combination), so residual variance is `sigma^2(1 + a^2 + b^2)` and `tau` grows by roughly `sqrt(3)`; detection power degrades faster than a single-query reading suggests, and **report how much detection power you lose**. "Output perturbation costs X% of integrity-detection power at noise scale Y" is a measured trade-off and precisely the kind of result this paper is built on. Two mechanisms that silently cancel is worse than either alone.

What linearity cannot catch, and the two cheap things that can

The gap. "A Cloud that answers consistently wrong in a way that respects linearity would evade this" is doing more work than it looks. The concrete instance to name is **a stale or wrong-index model**: a stale model is perfectly linear-consistent, so is the wrong cancer's key, and so is a uniformly scaled output. The linearity audit passes all three. And since functional keys are unrevocable and expiry is checked by the untrusted Cloud (§0.7 row 5), serving version 1 forever is not hypothetical — it is the default behaviour of a Cloud that never updates. Your own `hospital_vault` collision bug (§3) is exactly this failure class, so it is reachable in your own code. In a diagnostic pipeline a plausible number from the wrong model is the worst possible output, worse than an error.

Fix 1 — signed anchors, reinstated (~0.5 pd). I previously replaced pre-provisioned anchors with the linearity audit; that was an over-rotation. They are complementary: linearity verifies *internal consistency*, anchors verify you are consistent with the *right* model. At provisioning the Hospital ships a batch of signed `(test vector, expected score)` pairs alongside each sealed key; the Doctor injects one occasionally. Fresh ciphertext randomization means the Cloud cannot fingerprint the ciphertext, and because they are issued once at setup there is no per-query Hospital involvement, so §0.7 row 0 survives. *Two refinements:* anchors are effectively **single-use** — the returned score is fixed, so reusing one shows the Cloud the same output value twice, which is a fingerprint; provision a batch sized for the deployment lifetime. And under perturbation an anchor check needs the same tolerance treatment as the linearity check, for the same reason.

Fix 2 — cross-key consistency, free. If you adopt all-keys evaluation the Doctor already receives all 33 one-vs-rest scores per patient, and those are not independent: after calibration they should form a plausible distribution over tumour types. Two confident positives, or 33 near-identical values, indicates the Cloud is not evaluating 33 distinct correct keys. This needs no extra queries, no Hospital involvement and no audit budget, and unlike the linearity audit it *does* catch some wrong-key and stale-model cases, since a stale model for cancer `j` will disagree with the other 32. Nearly free given the calibration study in §4.2, and it partially covers the gap above. Statistical rather than deterministic — say so.

Deliverables: detection probability as a function of the audit rate `q` and the number of queries a cheating Cloud attempts; latency and cost overhead; and the indistinguishability argument. Note the one limitation honestly — a Cloud that answers *consistently* wrong in a way that respects linearity would evade this, so state the detection model as catching arbitrary or lazy corruption rather than a linear-consistent adversary.

Effort: 4 pd (R4), Week 6–7. Genuinely novel-adjacent, and it makes your system look like something a hospital could actually deploy. Cut this before you cut §5.1–5.4.

5.6 Documenting the `/model_metadata` leak

The endpoint returns exactly the Lasso-selected feature set. A malicious Doctor enumerates all 33 cancers and reconstructs the model's entire feature selection — meaningful IP — without ever seeing a coefficient. Under FHIPE this is the *only* remaining model leak, so it deserves proper treatment rather than the single line it currently gets.

Mitigations worth analysing (implement one, discuss the rest): serve a padded superset of indices so active genes hide among decoys; serve a fixed global index set across all cancers and let zero weights handle selection, which eliminates the leak at the cost of larger `n`; or rate-limit metadata queries per clinic. The first two are cheap and give you a leak-versus-latency trade-off figure. **Effort: 2 pd (R2).**

6. Workstream E — XAI, done so it does not leak

Your Section 7.2 design is an extraction oracle. The Cloud returns `weight_i × feature_i` per gene, and the Doctor already holds `feature_i` in plaintext — so it divides and reads off every weight. One diagnostic query, `k` divisions, entire model. It is a strictly *faster* attack than the one in §5.1.

Two other corrections before this goes in a paper. First, SHAP for a linear model is `weight_i × (feature_i - E[feature_i])`; mean-centring is not optional and omitting it will be flagged. Second, the "one of the first privacy-preserving XAI via chained FE" claim needs the §1 literature check to survive.

6.1 The fix: explain at pathway level

Group genes into biologically meaningful sets — KEGG pathways or MSigDB Hallmark gene sets — and issue **one query per pathway** rather than per gene. For each pathway `P`, the Hospital issues a functional key for the vector that is `w_i` on `i in P` and zero elsewhere. The Cloud returns the pathway's aggregate contribution.

Three things improve at once:

- **The leak closes.** Each returned value is a sum over many genes, so individual weights are underdetermined — one equation, many unknowns, no division attack. You can state the underdetermination precisely as a function of pathway size, which turns the fix into an analysable privacy result rather than a patch.
- **Clinical utility rises.** "DNA repair and cell-cycle checkpoint pathways dominate this patient's risk" is more actionable to an oncologist than a ranked list of 200 gene symbols. This is the honest justification, and it is a stronger one than the privacy argument.
- **Cost falls.** Roughly 30–50 pathway queries instead of 20–200 gene queries, and under FHIPE the per-patient `t[j]` vector (§2.3) is computed once and reused across all of them, so marginal query cost is far below the first query.

> **Check this before committing five days.** The claim that pathway aggregation leaves individual weights underdetermined depends on the *rank* of the pathway-by-gene incidence matrix, not on pathway size. KEGG and Hallmark sets overlap heavily, so if the rank equals or exceeds the number of active genes, the system of equations is fully determined and the leak is total — the privacy argument evaporates and only the clinical-utility argument survives. Building that matrix and calling `numpy.linalg.matrix_rank` takes ten minutes and decides whether §6 is a 5 pd contribution or a single honest paragraph. Do it first.

> **Pathway XAI and all-keys evaluation are close to incompatible.** Pathway explanations need one functional key per pathway for the *selected* model — roughly 30–50 keys. But if the Cloud must not learn which tumour type was selected (§5.4a), the explanation queries have to cover all 33 models too, i.e. 33 × 50 ≈ 1,650 key evaluations per explanation. That is not deployable. So you choose: reveal the tumour type to obtain an explanation and lose query privacy exactly at the moment the clinician most wants detail, or pay 33×. Say which, and note it in the decision guide — "explanations and query privacy are in tension under FE" is a genuine finding and it costs nothing to state.

Deliverables: pathway-level explanations for a handful of patients; a validation that pathway attributions agree with plaintext SHAP aggregated the same way; the underdetermination analysis; overhead measurement. Local rendering with matplotlib — keep the visualization simple, it is not where the marks are.

Effort: 5 pd (R3 + R1). Stretch goal — Weeks 7–8, only if FHIPE landed by Week 5. Tools: `gseapy` or plain MSigDB GMT files for gene sets, `shap` for the plaintext ground truth.

7. Reproducibility and inspectability

7.1 Folding inspectability in — four concrete deliverables

You asked how to make this part of the project rather than a talking point. Four artifacts, about six person-days total, and each one is something you can point at in a viva:

1. **A disclosure table in the paper.** One table, two columns: *published* versus *withheld*, with a justification per row. Published: training-cohort provenance and composition, feature-selection methodology, the active gene sets per cancer, intercepts, held-out and external performance, subgroup-stratified metrics, calibration curves, documented failure modes, version history, full training code. Withheld: coefficient magnitudes only. Making the asymmetry explicit and narrow is what makes the position defensible — a reader can see you are withholding one thing, not hiding behind a blanket claim.
2. **A model card, published as an artifact.** The above, filled in for your actual model. Follow the Mitchell et al. model-card structure. Include the random-feature projection matrix `W` if you adopt §10.1 — the Doctor computes `phi(x)` locally, so `W` is part of the model and must be versioned and distributed exactly like the weights. 2 pd (R3).
3. **A subgroup validation experiment.** This is the part that makes inspectability substantive rather than performative: report per-cancer F1 *and* performance stratified by whatever demographic covariates TCGA exposes, plus calibration per subgroup. If the model is worse for some group, that is exactly what a clinician needs to know and exactly what a coefficient dump would never tell them. It also folds neatly into your calibration study (§4.2), so the marginal cost is low. 2 pd (R3).
4. **An auditor endpoint, implemented.** Make §0.6 real rather than rhetorical: a `/audit` route, gated on an auditor client certificate, that issues the encryption capability plus functional-key access — i.e. deliberately grants the extraction capability that the Cloud is denied — and writes an entry to the hash-chained audit log. Roughly 2 pd once the certificate infrastructure from §3 exists, and it converts your most awkward finding into a demonstrated feature. This is the single cheapest novelty in the document.

Then write two paragraphs in the discussion saying plainly that diagnostic models *should* be inspectable by clinicians and regulators, that parameter disclosure is not what inspection requires, and that your architecture separates the two. You will be arguing a position you actually hold, which always reads better.

7.2 The reproducibility gap

Nothing in your repository produces `master_33_cancer_weights.npy`, `master_33_cancer_scaler.pkl`, or `master_33_cancer_model.pkl`. The pipeline cannot be reproduced from what you would submit. For a paper this is disqualifying at some venues and embarrassing at all of them.

Artifact	What it must do	Effort
`train_model.py`	Download or locate the Xena dataset, split with a fixed seed, train the one-vs-rest Lasso logistic regression, emit all three `.pkl`/`.npy` artifacts plus a metrics report. Record the seed and library versions.	2 pd (R3)
`requirements.txt` with pins	Exact versions, including the pairing library and its native dependencies. Note the Python version.	0.5 pd
`tests/`	At minimum: FHIPE correctness against plaintext dot products across random vectors and dimensions; serialization round-trip for every wire type; the extraction attack as a *regression test* that must fail under FHIPE and pass under Damgård; end-to-end three-node integration test.	3 pd (R1/R4)
`docker-compose.yml`	Three nodes plus certificate generation, one command to stand up. Solves your reviewers' install problem and your own demo-day risk.	1.5 pd (R4)
Model card / inspectability package	The artifact that makes your confidentiality position credible (§0.3): cohort provenance and composition, feature-selection methodology, external and held-out performance stratified by subgroup, calibration curves, documented failure modes, version history, intercepts, gene sets. Everything except coefficient magnitudes. Publish it alongside the paper.	2 pd (R3)
`bench/` + `README`	One script that regenerates every table and figure in the paper. Architecture diagram, threat model summary, honest limitations section.	2 pd (R4)

8. Schedule — four people, eight weeks

Two months of improvement work, then writing. Gates are the important part: they are the points where you decide to continue or fall back, on evidence.

Week	R1 — Crypto	R2 — Security	R3 — ML / Data	R4 — Systems
1	Backend abstraction; install petrelic *and* py_ecc; verify pairings work	Threat model; literature delta	Fix truncation + imputation bugs; retrain; `train_model.py`	Wire-format schema design; dead-code removal; repo hygiene
2	FHIPE on py_ecc; correct at n=8–32 vs. plaintext	**Extraction attack, measured** (both Cloud and Doctor paths); **global query budget + input-norm bounds**; Ed25519 identities; mTLS	Quantization sweep; range profiling	Latency harness; padding experiment
3	Petrelic backend; same test suite; **small-D wall benchmark on py_ecc**	Input validation; vault-collision fix; audit log; **hold-out retrain**	Quantization sweep complete; calibration study	Docker compose; CI running tests
GATE 1 (end of Week 3) — Re-pointed after §2.1 — correctness is no longer the risk, since PyMIFE's implementation is already verified correct. Gate on the two things that are: (i) ciphertext, functional key and public key round-tripping through JSON *between two separate processes*, and (ii) one working pairing on a fast backend (petrelic or mcl) agreeing with py_ecc at small n. Note the deliberately lowered bar: **correctness only, on the slow-but-certain backend, with no latency target** — performance and the petrelic install are Week 4–5 problems, not gate conditions. A gate that depends on a native-library install is a gate that fails for reasons unrelated to whether your scheme works. If NO → fall back to Route 4 (stay on Damgård; the query-budget work proceeds regardless) and reassign R1 to the random-feature non-linear study (§10.1), which is scheme-independent and would give Weeks 4–8 a strong centre.				
4	Delegated `ek`; multi-exponentiation; product-of-pairings	Sealed key delivery; delete `/fetch_key`; disclosure table	TenSEAL baseline; calibration by subgroup	Serialization for FHIPE types; integrate into nodes
5	Pollard kangaroo; per-cancer bounds; Design A blinding; exponent-noise construction; **signed anchors**	`/model_metadata` mitigation; `/audit` endpoint	Baseline table; subgroup validation	Three-node FHIPE end-to-end working
GATE 2 (end of Week 5) — full FHIPE pipeline running end-to-end? If NO → freeze features, spend Weeks 6–8 on measurement and writing. If YES → XAI is unlocked. **PERFORMANCE CHECKPOINT (same date, separate question)** — petrelic benchmarked at the intended n: is per-query time acceptable? Gate 1 deliberately dropped its latency target and Gate 2 asks only whether the pipeline runs, so without this the plan can reach Week 6 correct, integrated and unusably slow — and if petrelic slipped, Week 5 integration runs on `py_ecc` at 10–30 s per query with nothing flagging it. **Two thresholds, because one is a trap.** At end of Week 5 the system has baseline decryption, no all-keys, no perturbation-widened range and Design A barely begun — i.e. the *cheapest possible configuration*. Weeks 6–8 then add 33× pairings, a range widened by both blinding and the noise tail, and audit overhead; the gap between measured and deployed is plausibly 30–100×. So gate on *both*: (i) baseline per-query under N s, and (ii) **projected all-keys-plus-blinding-plus-noise under N s**, computed from §12's ×33 row and joint-range row — the multipliers are arithmetic you already have. State which configuration each threshold refers to. Without (ii) the checkpoint passes and the risk it exists to catch lands in Week 7. If either fails → freeze n at the largest feasible value, or drop all-keys, and report the reduced configuration explicitly.				
6	Optimization; scaling curves in `n`	Collusion demo; membership inference (optional); final threat-model table	Pathway grouping; plaintext SHAP; model card	**All-keys evaluation (4–5 pd); Designs A/B/C at 33 keys; trade-off table**
7	Pathway-level key generation	Paper: threat model + attack sections	Pathway XAI validation	Full experiment matrix; all figures
8	Paper: scheme + construction	Paper: security analysis	Paper: experiments	Artifact freeze; reproducibility check on a clean machine

> ***Capacity — state this number before you commit to the plan***
>
> Priced work in this document: Workstream A 12–18 pd, B 14–15, C ~11, D ~26 (up from 20 — all-keys evaluation at 4–5 and the re-priced result-privacy designs at 6–7 replace the old 2+4, plus 0.5 for anchors), E 5, §7 artifacts ~11, §10.1 4–5, §12 1, dimension decision 1. **Total core plan: 82–88 pd** (reduced by 3–6 after the §2.1 re-scope), plus §13's backlog at 11–13, so 96–107 pd overall.
>
> *Recomputed after this revision: all-keys evaluation was introduced as a mitigation without a price, and pricing it correctly added 6–8 pd. This is the second time the total has moved because a mechanism arrived unpriced — re-derive it whenever a new mechanism lands, not at the end.*
>
> Four people over eight weeks is 160 pd at full time, 96 pd at three days each per week, and **64 pd at two days each** — realistic for a capstone running alongside other modules. At two days the core plan now overruns by **18–24 pd** before the backlog is touched; at three days it exceeds 96 pd at the top of the range. The plan does not fit at realistic availability, and that is the number to confront rather than the prediction in §9.2.
>
> **Write down your assumed days per person per week, multiply it out, and put the total beside the 82–88.** If they do not meet, the two gates and §15's eight items already tell you what to cut — but nothing currently forces anyone to make that comparison, and "scope creep; nothing finished" is the only High-likelihood entry in the risk register. §9.2's "expect two-thirds of a plan this size" is a prediction; this is the version that is a decision.

Reserve Week 9 as slack. You will need it. If you do not, use it to tighten the paper.

9. Paper structure, honest positioning, and novelty

9.1 Suggested outline

The narrative arc that makes this land: *here is the natural architecture, here is what it silently leaks, here are the fixes, here is what each fix costs.* A paper with that shape has a result. A paper shaped "we implemented functional encryption for cancer prediction" does not. Concretely:

Section	Content
1. Introduction	Outsourced diagnostic inference; FE's distinct position in the design space (non-interactive, single-round, offline model owner) and the gap in systematic evaluation for genomics; the three inference-side properties a patient should have (§0.5); the observation that architectures of this shape deliver one and silently fail two; numbered contributions list. *State the exploratory framing here, in the first paragraph.*
2. Background	IPFE and the Damgård construction; function hiding and why public-key schemes cannot provide it (§0.1); brief FHE/MPC/TEE comparison — keep this shorter than your current draft does.
3. System and threat model	Three-node architecture; capability separation as the organizing principle; the coalition table (§5.3); **the paragraph establishing TCGA as a reproducible surrogate for a private deployment cohort** (§0.4). Do not omit that paragraph.
4. Attack I: weight extraction	The chosen-plaintext oracle, measured across 33 models: queries, wall-clock, exactness, surrogate-model fidelity. Optionally the training-cohort leakage chain (§5.2), clearly labelled as a surrogate measurement.
5. Attack II: what the Cloud learns	Query privacy via dimension fingerprinting, and result privacy — the Cloud obtains the diagnosis. Quantify both (§5.4).
6. Construction	FHIPE deployment; delegated encryption key `g1^B` (§2.3); sealed key delivery (§2.8); blinding Designs A, B and C (§0.5); padding. State the trade-off as a proposition scoped to the offline-model-owner setting.
7. Evaluation	Quantization–accuracy–calibration; non-linear extension via random features and the expressiveness/cost wall (§10.1); inner-product range profiling; latency breakdown and scaling in `n`; the non-amortizable discrete log (§2.4); result-privacy trade-off curves; CKKS and plaintext baselines.
8. Transparency	Disclosure table, model card, subgroup validation, the `/audit` endpoint and tiered disclosure (§7.1). Short section, distinctive, and few comparable papers have one.
9. When to use this, and when not	The payoff of the exploratory framing and, for many readers, the most useful section in the paper. A decision guide: FE suits linear models, modest dimensions, an offline model owner, and a genuinely non-colluding evaluator. It does not suit non-linear models at all, nor settings where the evaluator may collude with the client, nor workloads where the model owner is online anyway (use MPC or Paillier — they will be faster). Include what §10 establishes: kernel machines are reachable via random feature maps at a measured dimension cost, small networks are reachable layer-wise with documented activation leakage, and tree ensembles are out of reach at any degree. Two pages, mostly prose, and it is what saves the next group their eight weeks.
10. Limitations	Doctor–Cloud collusion; linear-model constraint; non-post-quantum; curve security level if you ship BN254; residual `/model_metadata` leak. Volunteering these buys credibility — a reviewer who finds them unlisted discounts everything else.
11. Related work / 12. Conclusion	The Week-1 literature delta becomes your related-work section, so it is written once and used twice.

On venue, honestly: this is workshop-to-mid-tier material, not a top-tier security conference, and that is a perfectly good outcome for a capstone. Privacy or applied-crypto workshops and health-informatics venues are the realistic targets; an arXiv preprint plus a workshop submission is a sensible default. Aim the writing at a reviewer who knows cryptography but not oncology, and define your TCGA abbreviations.

9.2 Honest assessment of where this lands

If executed, this is a publishable paper at a **workshop or mid-tier venue**. It is not a top-tier security conference paper, and it is worth being clear-eyed about why: no single contribution here is large. There is no new cryptographic construction, no surprising theoretical result, and the central attack is folklore made concrete.

But under the exploratory framing on page 1, that is the correct shape rather than a shortfall. Feasibility and design-space papers are not judged on whether you invented something; they are judged on whether the evaluation is thorough, honest, and reusable by the next group. That is a bar four people can clear in eight weeks, and it is a far better fit than trying to invent cryptography on a deadline. It also promotes two things from side-deliverables to primary contributions: the **artifact** (a working Python FHIPE and a reproducible benchmark harness will be used by people who never read your paper) and the **decision guide** (§9.1, section 9). Aggregate, this is well above the bar for a final-year capstone.

> **One caution about under-explored areas.** Verify that the gap is real — and if it is, ask why. Sometimes a design point is under-explored because it is genuinely unpromising, and here there is an obvious candidate reason: IPFE evaluates inner products only, so it is confined to linear models, while genomic ML is moving toward non-linear ones. Section 10 is your response to that objection — it pushes the boundary outward and, more importantly, locates it precisely. Your honest conclusion may well be "*this design point is viable but narrow, and here are its precise boundaries*." Do not fight that conclusion if the evidence points there. A bounded negative result from a careful evaluation genuinely saves the next group months, which is exactly the contribution you said you wanted to make — and reviewers reward it far more reliably than a strained positive claim.

The five things that most raise or lower where it lands, in order of leverage:

1. **Whether you can name published systems with the vulnerable pattern** (§1). "The naive design is broken" is weak. "These three published architectures are broken" is a finding. Same experiment, different standing.
2. **Whether you keep the security claims inside what you can support.** The `ek` modification is not covered by the KLMMRW proof (§2.3), the result-privacy trade-off is not an impossibility (§0.5), and FE revealing the function value to its decryptor is definitional rather than a discovery. Overclaim any of these and the reviewer discounts the rest.
3. **Whether the offline-model-owner argument is made explicitly** (§0.7, row 0). Without it, "why not Paillier or secret sharing?" has no answer and the whole approach looks unmotivated.
4. **Whether the artifact actually runs.** A working Python FHIPE with serializable keys will be used and cited by people who do not read your paper. Entirely within your control, and under the exploratory framing it is a primary contribution rather than a nicety.
5. **Whether you write the decision guide.** Two pages of honest "use this when, avoid it when" is the single most reusable thing you can produce, and it is the natural payoff of an exploratory study. Cheap, and most papers omit it.

Realistic execution expectation: four undergraduates over eight weeks land perhaps two-thirds of a plan this size. Two-thirds of this is still a paper — which is the point of the gates in §8 and the priority ordering in §15. Protect the measurement work first; it survives any FHIPE outcome.

9.3 Novelty ledger — what to claim, and how strongly

Be precise here. Overclaiming is the fastest route to rejection, and you have enough genuine material that you do not need to.

Contribution	Strength	How to phrase it
Practical weight-extraction attack on public-key IPFE inference deployments, measured on 33 real TCGA models	**Strong**	Frame as a deployment-security result, not a break of Damgård. The theory is folklore; the *measurement on a realistic genomic pipeline*, and the observation that a published architecture pattern is vulnerable, is not. Lead with this.
Result privacy: the Cloud learns the diagnosis — the unexamined leak of §0.5, with three fixes and a characterized trade-off space	**Moderate–strong**	Build the abstract around this. Inference-side, patient-centred, and present in any architecture of this shape. Note honestly that *FE reveals the function value to whoever decrypts* is definitional and well known — your contribution is the architectural observation that the compute node need not be the decryptor, plus three designs with measured costs. Scope the claim to the offline-model-owner setting (§0.5).
Query privacy: the tumour type under test leaks — via ciphertext dimension, and via key selection under long-lived keys (§5.4a)	**Moderate**	Promoted from footnote. Frame as leakage of which primary site is being investigated for a named patient, plausibly more sensitive than the expression values the system was built to protect. The countermeasure is *all-keys evaluation*, not padding, and it trades suite lifetime for query privacy at a rate of roughly 33 (§1) — report the exchange rate, which is the interesting part.
Weight recovery to training-cohort leakage on a public surrogate cohort (§5.2)	**Modest**	Optional supporting measurement. State clearly that TCGA is a surrogate for a private deployment cohort and that no harm to TCGA participants is implied. Cite Fredrikson et al. and Homer et al. for precedent.
Capability-separation architecture: extraction reduced from a free offline operation to an online, logged, rate-limited one, with a per-coalition statement	**Moderate**	Your main design contribution, and the answer to "why is there a Cloud node." Phrase it as changing the *cost and visibility* of extraction, not as preventing it — the Doctor-side path is inherent (§0.1). Justify confidentiality from the deployment setting, never from IP.
Tiered disclosure — the same capability partition denies extraction to the compute node while granting it to credentialed auditors, with an implemented `/audit` endpoint (§0.6, §7.1)	**Moderate**	Cheap to state, memorable, and it dissolves the transparency-versus-confidentiality objection instead of arguing with it. Implement the endpoint rather than only describing it — two person-days turns a rhetorical move into a demonstrated feature. Present as a policy layer over the crypto, not a cryptographic guarantee.
Delegated encryption key `g1^B` —	**Moderate**	Small but real construction result. Report the `O(n^2)` cost honestly

encryption capability without master-secret distribution		alongside the multi-exponentiation and sparsity mitigations.
Serialization and a performant backend for an existing Python FHIPE that could not previously cross a process boundary (§2.1)	**Moderate**	Artifact contribution, now precisely bounded by measurement rather than assumption. **Do not claim "first Python FHIPE"** — PyMIFE's exists and is correct. Claim the gap you actually close: ciphertexts do not serialize, `export()` is unimplemented, and the only backend needs ~12 minutes per query at `n`=200. That the same serialization gap appears in both GoFE and PyMIFE is itself worth a sentence about the state of distributed FE tooling.
Non-amortizable discrete-log cost of function hiding (§2.4, Finding 1)	**Moderate**	A crisp systems insight: the standard precomputation optimization for IPFE decryption is unavailable under FHIPE. Nice because it is counterintuitive and cheap to demonstrate.
Quantization–accuracy–calibration characterization for integer-domain FE on 33-class TCGA	**Moderate**	Empirical, useful to anyone building on IPFE. The calibration angle — error is worst where decisions are marginal — is the freshest part.
Query-pattern (dimension) leakage and padding countermeasure	**Modest**	One figure, one paragraph. Include; do not headline.
Probabilistic result-integrity auditing for untrusted FE compute	**Moderate** if completed	Under-addressed in FE inference work. Frame as lightweight verifiability, explicitly contrasted with full verifiable FE.
Non-linear inference under a linear FE primitive via random feature maps, with the expressiveness/dimension/latency boundary measured (§10.1)	**Moderate–strong** if completed	Directly rebuts the "IPFE means linear models only" limitation that otherwise bounds your whole design point. Cheap, needs no new cryptography, and locating the practical wall is precisely the kind of boundary result the exploratory framing calls for. Check the literature first.
Pathway-level FE explanations resisting weight recovery	**Moderate** if completed	Only claim if the underdetermination analysis is done. The clinical-utility argument is the honest primary justification; privacy is the bonus.
Query budgets and calibrated noise as the defence against the inherent Doctor-side extraction path, with the extraction-error versus diagnostic-accuracy curve	**Moderate**	Required rather than a fallback (§0.1), but the framing must be exact: **the budget bounds rate and the log provides attribution — neither prevents extraction**, because `n` legitimate patients suffice. Dimension is the only lever that scales. Perturbation is reported as a *measured characterization* and not a formal privacy guarantee (decided framing). The exponent-noise construction — an untrusted evaluator perturbing a value it cannot read — is the part worth a literature check, but **rate its novelty low**: mechanically it is Design A with the Cloud as the actor instead of the Hospital, and blinding in the exponent is a textbook technique. The *framing* is mildly interesting; the construction is not new. Report it as an engineering choice with a nice property, not as a contribution. If FHIPE fails at Gate 1 this becomes the headline instead.

> **Do not claim:** that the system protects the model unconditionally; that weight confidentiality is desirable for its own sake (argue it from training-cohort privacy, or drop it); that confidentiality and clinical inspectability are in tension (they are not — §0.3); that Damgård IPFE is function-hiding "in practice"; that per-gene FE-SHAP preserves privacy; "first privacy-preserving XAI via FE" without the literature check; that TCGA participants are put at risk by weight recovery (they are not — §0.4); that the Cloud learns nothing about the patient (it currently learns the diagnosis — §0.5); that model confidentiality holds against the Doctor (it cannot — §0.1); that FE in general cannot be function-hiding, rather than scoping the statement to the inner-product class; that perturbation gives a differential-privacy guarantee (you decided to report it as a measurement); that the query budget prevents extraction (it bounds rate only — §0.1); that extraction requires deviation from protocol (`n` ordinary patients suffice); that the model predicts cancer *risk* when it classifies tumour *type*; that yours is the first Python FHIPE (§2.1 — it is not); or 128-bit security if you ship on BN254. Each of these is checkable in minutes by a reviewer, and one bad claim colours the reading of everything else.

10. Pushing past linear models

"IPFE only does inner products, therefore only linear models" is the sharpest limitation in your project and the most likely reason a reviewer calls the design point narrow. It is also less binding than it looks. Three routes, ranked; the first requires no new cryptography whatsoever.

10.1 Random feature maps — non-linear models under linear FE **BEST OPTION**

The constraint is that the *evaluation* must be an inner product. Nothing requires the inner product to be taken over the raw features. Random Fourier features (Rahimi & Recht) give an explicit map `phi` such that `<phi(x), phi(z)>` approximates an RBF kernel, and `phi` is **data-independent and public** — it is just a fixed random projection followed by a cosine.

So: the Doctor computes `phi(x)` locally in plaintext, encrypts *that*, and the Hospital's model is a weight vector over the `D` random features. One IPFE query, unchanged protocol, unchanged scheme — and the model being evaluated is a kernel machine, not a linear one. Non-linear inference under a linear evaluation primitive, with zero cryptographic change.

> **Which scheme this runs on, stated up front.** FHIPE `Setup(D)` inverts a `D×D` matrix (`O(D^3)` field operations) and delegated encryption is `O(D^2)` group exponentiations, so the dimensions this experiment wants are out of reach: at `D = 5000` that is roughly `10^11` field operations, a 1.2 GB encryption key and about `2.5×10^7` exponentiations. The realistic FHIPE wall is around **`D = 200–500`**. So the sweep runs on plain Damgård IPFE, where setup and encryption are both linear in `D`; FHIPE is demonstrated only at the low end and its wall is reported as a separate measured result. Say this in the paper rather than letting a reader discover the sweep quietly changed schemes. It also means this experiment is a *sound* Gate-1 fallback — it does not depend on the thing that failed the gate.

Note the collision with §1. *Random features and the union-dimension decision draw on the same budget — the dimension you can afford — and both want to spend all of it. If you adopt `n_union` at, say, 800, you are already past the FHIPE wall and the random-feature sweep has nothing left to spend; conversely a `D` large enough for a decent kernel approximation precludes the union. Under FHIPE these are mutually exclusive, and you should pick one deliberately in Week 1 rather than discovering the conflict in Week 6. Under Damgård both are affordable, which is a further argument for running the dimension and random-feature experiments on Damgård and reporting the FHIPE wall as the boundary.*

Benchmark the FHIPE dimension wall in Week 3, as part of the Gate-1 evidence: time `Setup`, `ek` size and encryption at `D` = 64, 128, 256, 512, 1024. That single table determines what is feasible for padding (§5.4a), for random features, and for the base system, so you want it before you commit Weeks 4–8.

The experiment: train RBF-kernel logistic regression (or an SVM) on the TCGA task via random features, sweep `D` from 100 to 5000 under Damgård IPFE, and report accuracy against both the linear baseline and the exact kernel model, alongside encryption cost, ciphertext size and end-to-end latency at each `D`. The trade-off is the finding: expressiveness costs dimension, dimension costs everything else, and there is a point where FE stops being practical. Locating that point precisely is exactly the kind of boundary your design-space map should contain. Report the FHIPE-feasible sub-range separately, so the accuracy-versus-dimension curve and the scheme-feasibility boundary are both visible on the same axes.

Effort: 4–5 pd (R3 with R4). Check the literature first: random features under FE is a natural enough idea that it may exist, though I am not aware of it for genomic inference. Even if it does, the measurement on your task is still worth reporting. *This is also the natural substitute deliverable if FHIPE fails at Gate 1* — it is independent of the scheme and would give Weeks 4–8 a strong centre.

10.2 Layer-wise neural network evaluation

A small MLP can be evaluated with the IPFE you already have, one layer at a time: the Cloud computes each neuron's pre-activation as an inner product (one functional key per neuron), the Doctor decrypts the layer, applies the non-linearity locally, re-encrypts, and the next layer proceeds. Weights never leave the Hospital. Costs one round trip per layer and `k` queries per layer of width `k`.

The honest caveat, which is itself worth reporting: the Doctor learns every intermediate activation, and with chosen inputs that is a straightforward path to weight recovery — the same capability argument as §0.1, now available layer by layer. So this is a demonstration of *feasibility with a documented leakage cost*, not a secure construction. A two-layer network with 32 hidden units on the Lasso-selected genes is a perfectly good demonstration. **Effort: 5–6 pd.** Do 10.1 first.

10.3 Quadratic functional encryption

Schemes for degree-2 functions exist — the Baltico–Catalano–Fiore–Gay line of work, with the Sans–Gay–Pointcheval construction implemented in GoFE. Degree 2 buys real expressiveness: quadratic discriminant analysis, factorization-machine-style interaction models, and a one-hidden-layer network with squared activation, which is exactly a degree-2 polynomial in the input. Applied to the 20–200 Lasso-selected genes rather than all 20,000, the `n^2` term stays tractable.

The catch is that this is a second new scheme, in Go, with the same serialization problem you already hit. Treat it as future work with an optional small demonstration, not as a Week-6 commitment. **Effort: 8–12 pd if attempted.**

10.4 The boundary: what stays out of reach

Decision trees, random forests and gradient-boosted models require comparisons, which inner-product FE cannot express at any degree — you would need order-revealing encryption or garbled circuits, i.e. a different primitive and an interactive protocol. Say this explicitly in the decision guide. Knowing precisely where an approach stops working is a genuine contribution, and it is one most papers decline to make.

11. Other future work

- **Multi-Input FE for multi-modal fusion — probably cheaper than you think.** You have this filed under future work as blocked by pairings and GoFE serialization. Check that assumption: DDH-based multi-input IPFE constructions (Abdalla, Catalano, Fiore, Gay & Ursu, 2018) are built *from single-input IPFE plus secret sharing* and need no pairings at all, and PyMIFE ships multi-client modules alongside the single-input Damgård you are already using. Spend half a day verifying what those modules actually support before writing this off — if they work, cross-institution fusion is a week's work rather than a rewrite, and it is the strongest justification for the Cloud's existence (§0.7, #2 and #3). Under Route II this becomes your centrepiece rather than future work.
- **Cross-institution encrypted aggregation.** The cheapest way to make the Cloud indispensable: have it sum encrypted per-patient scores across sites to produce cohort statistics no single hospital can compute. Even a two-hospital demonstration substantiates the architecture. Roughly 3 pd if the multi-client modules cooperate.
- **Verifiable FE.** The principled version of §5.5. Cite as the proper solution and position your audit as the pragmatic approximation.
- **Post-quantum (LWE-based) IPFE.** Discussion only. Library support is thin and it is a project in itself. Do not promise an implementation.
- **Multi-hospital federation.** The strongest deployment story for the Cloud node: an untrusted broker serving many hospitals and many clinics, avoiding N×M key distribution. You can motivate it in prose without implementing it.
- **Adaptive search bounds.** Have the Hospital ship a per-cancer bound inside the sealed key blob, derived from your range profiling, rather than hardcoding a global `L`. Small, and it makes the profiling experiment actionable rather than merely descriptive.
- **Formal security statement.** Even a half-page IND-CPA-style game for the capability-separated setting, without a full proof, substantially raises how the paper reads. If R1 finishes FHIPE by Week 5, this is a better use of Week 7 than more optimization.

12. The one-page budget table — build this before the next revision

Almost every arithmetic error found in review so far — the blinding-width ceiling, the gate's latency target, the random-features dimension wall, the `ek` size at the padded operating point — exists because this document reasons in asymptotics and prose where it needs absolute magnitudes at the target dimension. One table fixes that class of defect permanently. Fill the estimate column now from arithmetic, and replace each cell with a measurement as it arrives; **the gap between the two columns is itself a reportable finding.**

Quantity	n=64	n=128	n=200	n_max (per-cancer)	n_union (F4)	Notes
`Setup` time (matrix inverse, `O(n^3)`)						Per dimension, one-off. Dominates provisioning.
`ek` size (`n^2` G1 points)						**Two curves, two numbers.** Compressed G1 is 32 B on BN254 (petrelic, your performance backend) → 1.28 MB at n=200; 48 B on BLS12-381 (py_ecc) → 1.92 MB. Quote the one matching the row.
Encryption time (`n^2` exps, with and without multi-exp)						The number that decides feasibility.
Ciphertext size						Linear in `n`.
Functional key size						Linear in `n`.
Pairing product time (`n` pairings)						Report with and without a product-of-pairings primitive.
Discrete-log time at profiled range `L`						Not amortizable under FHIPE (§2.4).
Same, at Design A blinding `k` = 8 / 16 / 20						Establishes the hard ceiling on `k`.
Same, with perturbation tail included						Truncated noise widens `L` too (F7).
Same, **with blinding and perturbation together**						The case you intend to deploy. Both feed one `O(sqrt(L))` cost, so the joint figure is what pushes the `k` ceiling down further. Currently unbudgeted anywhere.
Pairing time × 33 (all-keys evaluation)						The query-privacy mitigation of §5.4a. Multiplies the pairing row by 33.
End-to-end wall clock						py_ecc and petrelic, separate rows.

Effort: 1 pd for the estimate column (R1), then incremental. Do it in Week 1. It is also the raw material your decision guide is built from, so it is not overhead.

13. Tracked backlog — real items not yet folded into the plan

These are all genuine and none are in the schedule above. Recorded here rather than silently dropped, roughly in priority order. Two former entries are struck through, having been folded into the plan proper. **Remaining total: about 11–13 pd** — a menu, not a commitment.

Item	What and why	Cost
Accuracy versus `n` sweep	`n` drives every cost in the system, yet you have no curve relating it to accuracy. This is the single figure the decision guide is built from — how much accuracy does each gene buy, and where does the curve flatten.	1 pd (R3)
TCGA data-use and cohort realities	No statement of data tier or access terms; nothing on batch effects and cross-platform transfer, which is the main reason genomic ML fails to deploy; nothing on the roughly 30× class imbalance between BRCA and CHOL sitting under a macro-F1 headline. A health-informatics reviewer raises all three.	2 pd (R3)
Random features may lose to linear	Kernel approximation error falls as `1/sqrt(D)`, so at the FHIPE-feasible `D` the kernel model may underperform the plain linear baseline. Predict this before running it, and add Nyström as a comparison point. A negative result here is fine but should not be a surprise.	0.5 pd to predict
Decryption time is a side channel	Discrete-log search duration depends on the value recovered, so response latency leaks information about the score — including under Design A, where blinding widens the range but does not flatten the timing. Cheap to measure and it belongs in §5.4 beside the other inference-side leaks.	1 pd (R4)
`t[j]` caching invariant	Reusing the per-patient vector across queries (§2.3, mitigation iii) is a randomness-reuse hazard if the per-query mask is ever reused with it. State the invariant explicitly and write a named test that fails if it is violated.	0.5 pd (R1)
Auditor revocation and exposure	The `/audit` channel has no revocation story, and `N` auditors multiply extraction exposure linearly. Needs a certificate-revocation path and a stated cap on concurrent auditors.	0.5 pd (R2)
External cohort named	§7.1 promises external validation performance; no external cohort is currently identified. Either name one (GTEx, ICGC, a GEO series) or drop the promise.	0.5 pd (R3)
Walking-skeleton milestone	Three-node end-to-end currently first appears in Week 5, immediately before Gate 2. Stand up a trivial end-to-end path in Week 2 with stub crypto so integration risk surfaces early rather than at the gate.	1 pd (R4)
Two-hospital aggregation demo	§0.7 rates cross-institution aggregation a *strong* justification for the Cloud, but nothing is built. Even a two-site demonstration substantiates it.	3 pd (R4)
~~Padding enables a single~~ `Setup`	*Now in the plan* — folded into §1's dimension decision as consequence (i). Removed from the backlog total.	—
~~Extraction metrics~~	*Now in the plan* — folded into §5.1's deliverables. Removed from the backlog total.	—
Writing schedule	Nothing is written before Week 7, then four people write simultaneously in Week 8. The threat model, literature delta and decision guide are already prose — draft them as paper sections when they are produced, not later.	Reschedule
Risk register gaps	Missing: team availability; ML accuracy too weak to be credible (set a macro-F1 floor now, before you are invested); an existing Python FHIPE invalidating the artifact claim; insufficient writing time.	0.5 pd
Glossary and housekeeping	Glossary lacks random feature map, Pollard kangaroo, mTLS and Designs A/B/C. No secrets-at-rest item in §3. No artifact licence or DOI. No threats-to-validity subsection.	0.5 pd

Consistency procedure — extend it when you add a mechanism

The load-bearing-claim list catches wrong *content*. Most defects found in the last two review rounds were not wrong content but **wrong seams**: two individually-correct mechanisms colliding over a shared resource. So add one step. When you introduce a mechanism, write down every other mechanism it shares a resource with, and check each.

Mechanism	Shares a resource with
All-keys evaluation	The pairing budget (performance checkpoint); suite lifetime (dimension decision); the query counter (granularity becomes per `(model, key)`); the key count (Designs A/B/C); the dimension budget (requires a *common* dimension — it is a prerequisite, not an independent choice); pathway XAI (33× or reveal the type).
Common dimension	FHIPE feasibility wall; random features (both spend the same dimension budget, mutually exclusive under FHIPE); `/model_metadata`; extraction lifetime.
Output perturbation	The dlog range and hence latency; the loud out-of-range rule; the linearity audit tolerance; the anchor check tolerance; Design A's `k` ceiling.
Long-lived keys	Query privacy (key selection leaks); revocation and model versioning; the stale-model integrity gap.
Any new priced item	The capacity total in §8. Re-derive it on the spot.

14. Risk register

Risk	Likelihood	Mitigation
Pairing library will not install or lacks GT exposure	Medium	Test *both* petrelic and py_ecc on day 1, before any scheme code. py_ecc is pure Python and cannot fail to install. The backend abstraction means a library swap is a day, not a rewrite.
FHIPE too slow to be credible	Medium	Range profiling shrinks the discrete-log search; product-of-pairings and multi-exponentiation address the rest. Worst case, report honest numbers at a reduced `n` and discuss the gap — a measured "FHIPE costs 40× Damgård here, and here is why" is a legitimate result, not a failure.
Scheme implemented but subtly wrong	Medium	Property-based tests against plaintext dot products over random vectors and dimensions, on both backends, from day one. Never trust a single hand-checked example.
Novelty claim already published	Medium	Literature delta in Week 1, not Week 8. You have several independent contributions, so losing one is survivable — losing all of them in Week 8 is not.
Scope creep; nothing finished	**High**	The two gates. Ship the empirical workstreams (§4, §5.1–5.4) early — they alone support a paper. Treat XAI and result-integrity auditing as genuinely optional and be willing to cut them.
Uneven workload; R1 becomes the bottleneck	Medium	Workstreams B, C, D are independent of the FHIPE outcome and account for the majority of your figures. R4 joins R1 from Week 4 once the wire format is settled.

15. If you only do eight things

1. **Fix the truncation and imputation bugs** (Week 1) — every number you report depends on them, and one is probably costing you accuracy right now.
2. **Write the threat model** (Week 1), including the paragraph that says TCGA is a reproducible surrogate for a private deployment cohort (§0.4). That paragraph pre-empts the strongest objection to your whole premise.
3. **Close the result-privacy gap** (§0.5, §5.4b) — your Cloud currently learns the diagnosis. Implement all three fixes, measure the trade-off space. This is the best new contribution available to you and it is squarely inference-side.
4. **Measure the extraction attack** (Week 2) — still the highest value per person-day, and now framed as what forces the architecture rather than as an IP concern.
5. **Take the dimension decision** (§1) — five minutes to compute the global active-gene union, and it is your only defence that scales, plus it closes three leaks. Then ship the budget and log as rate-and-attribution controls, and plot patients-until-recovery against noise and against accuracy loss. That figure is the paper's centre.
6. **Promote query privacy to a real result** (§5.4a) — note the mitigation is now all-keys evaluation, not padding, and it trades against model lifetime. Cheap, and it completes the set of three inference-side properties.
7. **Make PyMIFE's existing FHIPE usable** (Weeks 2–5) — serialization and a fast backend, not a from-scratch implementation (§2.1) — with a real gate at Week 3, plus sealed key delivery.

8. **Build the inspectability deliverables** (§7.1) — disclosure table, model card, subgroup validation, and the `/audit` endpoint. Six person-days, and it lets you hold your actual position: models should be inspectable, and parameter disclosure is not what inspection means.

The shape of the paper is now clear and it matches what you set out to build. It is about **inference-side privacy in outsourced diagnostic computation**: three properties a patient should have, an architecture that delivers one of them and silently fails the other two, the fixes, and what each fix costs. Model confidentiality is a supporting requirement — justified by the deployment setting, bounded by an honest impossibility statement, and paired with a transparency story you actually believe. That is a coherent contribution, it is reachable in eight weeks with four people, and none of it requires you to defend a claim you would rather not make.

16. Verification tasks and reading list

Four claims in this document could still be wrong in ways that cost you weeks. Each is under a day to settle. Two are done; two are yours. This section also gives the literature you need, because "we did not know where to look" is a fixable problem and an unread related-work section is the most common reason a capstone paper is rejected.

16.1 Status of the four checks

Check	Status	Result or how to run it
1. Gene union (§1)	**Yours**	Run `check1_gene_union.py master_33_cancer_weights.npy`. Prints per-cancer active counts, the global union, both extraction-lifetime metrics, FHIPE cost at that dimension, and which branch of the §1 decision tree you are in. Five minutes once the weights file exists.
2. Pathway rank (§6.1)	**Yours**	Run `check2_pathway_rank.py <msigdb.gmt> <active_genes.txt>`. Download the Hallmark or KEGG GMT from MSigDB (`gsea-msigdb.org`, free registration), write your active gene symbols one per line. If rank ≥ number of active genes, §6's privacy argument is void and the section becomes one paragraph. Watch the symbol namespace — HGNC symbols versus Ensembl IDs is the usual reason nothing matches.
3. Existing Python FHIPE	**Done**	Settled, and it changed the plan — see §2.1. PyMIFE has one and it works; charm-crypto does not have one at all. Verified by installing and running both.
4. Published vulnerable systems	**Yours**	Needs literature search, which cannot be automated from here. Method in §16.2. Two person-days, and it is the highest-leverage task in Week 1.

16.2 Check 4, in plain terms

What you are looking for. A published system that uses a *public-key* inner-product FE scheme for machine-learning inference, and gives one party both the public key and a functional key. That party can then recover the model (§0.1). You want to know whether anyone has actually built and published such a thing.

Why it matters. If you find none, your central result reads "the obvious design is broken" — true, but folklore, and a reviewer may say so. If you find two or three, it reads "these published architectures are vulnerable, and here is the measurement." Same experiment, same two days of work, substantially different standing. This single task moves the paper more than any other item in Week 1.

The tell, concretely. Open the paper's architecture diagram and answer three questions:

1. Is the scheme *public-key* IPFE — Abdalla et al., Agrawal et al., "DDH-based IPFE," anything not explicitly described as secret-key or function-hiding? If it is function-hiding or secret-key, move on; that design is fine.
2. Does some party other than the model owner perform the `Decrypt` operation? Look for a node labelled cloud, server, evaluator, or analyst.
3. Does that same party hold, or have access to, the master public key? In a public-key scheme it does by definition — the key is public.

All three yes means the pattern is present. Record the paper, the scheme, the party, and the section or figure number. Three or four entries is enough for a paragraph in related work.

Do not overclaim what you find. Write "the architecture as described admits weight recovery by the evaluating party," not "system X is broken." Some papers state the limitation in a threat model you have not read carefully — check before asserting. If a paper *does* note it, that is still useful to you: cite it as prior awareness and position your contribution as the measurement rather than the observation.

16.3 Where to search, and with what

Resource	Use
IACR ePrint `eprint.iacr.org`	Free, complete for cryptography, full text searchable. Start here. Almost every FE paper appears here first.
Semantic Scholar `semanticscholar.org`	Best "cited by" graph and a free API. The key move: open the KLMMRW paper (ePrint 2016/440) and the Abdalla et al. IPFE paper, then read the *citing* papers. Applied systems cite the schemes they use, so this finds them.
DBLP `dblp.org`	Reliable author and venue listings. Use it to find everything by a group once you identify one relevant paper.
Google Scholar	Broadest coverage, noisiest. Good for the applied and medical-informatics literature that never reaches ePrint.
Connected Papers `connectedpapers.com`	Visual citation neighbourhood from one seed paper. Fast way to see whether a subfield exists.

Search strings that work. Combine one from each column and vary:

- *Scheme:* "inner product functional encryption", "IPFE", "function-hiding inner product", "functional encryption"
- *Application:* "inference", "classification", "machine learning", "logistic regression", "neural network", "prediction", "diagnosis", "genomic"
- *Architecture:* "outsourced", "cloud", "delegated", "three-party", "untrusted server"

Also search the negative space, which is how you check your own novelty: "functional encryption" + "model extraction", "IPFE" + "leakage", "functional encryption" + "differential privacy". If someone has already made your central observation, you want to find that in Week 1 and cite it, not in Week 8 from a referee.

16.4 Reading list

> **Verify every one of these before citing.** They are given from memory, and although the high-confidence entries are ones I am fairly sure of, titles, years, venues and author lists can be misremembered. Check each on ePrint or DBLP and read at least the abstract before it enters your bibliography. A citation that does not exist, or does not say what you claim, is worse than no citation — and it is the single easiest thing for a reviewer to catch. Treat this as a search index, not a bibliography.

Tier 1 — read these properly (the eight that matter most)

Paper	Venue	Conf.	Why
Kim, Lewi, Mandal, Montgomery, Raykova, Wu — *Function-Hiding Inner Product Encryption is Practical*	SCN 2018 ePrint **2016/440**	High	Your scheme. PyMIFE's module cites this exact number, so the ePrint ID is confirmed. Read §2–3 closely; it is what §2.2 restates.
Abdalla, Bourse, De Caro, Pointcheval — *Simple Functional Encryption Schemes for Inner Products*	PKC 2015	High	The foundational DDH-based IPFE. This is the family your current system uses and the one the extraction attack applies to.
Tramèr, Zhang, Juels, Reiter, Ristenpart — *Stealing Machine Learning Models via Prediction APIs*	USENIX Security 2016	High	Cite this for the §0.1 lifetime result. Establishes that a linear model falls to roughly `n` prediction queries. Your contribution is the FE-deployment framing, not the attack.
Dufour-Sans, Gay, Pointcheval — *Reading in the Dark: Classifying Encrypted Digits with Functional Encryption*	ePrint 2018/206	High	**Start check 4 here.** FE for classification by the authors of the scheme literature. Most likely single candidate for the vulnerable pattern.
Marc, Stopar, Hartman, Bizjak, Modic — *Privacy-Enhanced Machine Learning with Functional Encryption*	ESORICS 2019	Med-high	The GoFE/CiFEr authors (XLAB) applying FE to ML. Second candidate for check 4, and relevant to your library discussion.
Ligier, Carpov, Fontaine, Sirdey — *Information leakage analysis of inner-product functional encryption based data classification*	PST 2017	Medium	**Read early.** The title suggests it may already contain part of your leakage argument. Better to know in Week 1.
Fredrikson, Lantz, Jha, Lin, Page, Ristenpart — *Privacy in Pharmacogenetics: warfarin dosing*	USENIX Security 2014	High	Model inversion on a linear genomic model. The precedent for §0.4's framing.
Rahimi & Recht — *Random Features for Large-Scale Kernel Machines*	NIPS 2007	High	§10.1 rests on this. Short and readable; note the `1/sqrt(D)` error rate, which is what predicts your possible null result.

Tier 2 — know what they say; read the abstract and the relevant section

Paper	Venue	Conf.	Relevant to
Bishop, Jain, Kowalczyk — *Function-Hiding Inner Product Encryption*	ASIACRYPT 2015	Med-high	Function-hiding definitions; §2.3's security-claim boundary.
Agrawal, Libert, Stehlé — *Fully Secure FE for Inner Products from Standard Assumptions*	CRYPTO 2016	High	Background; LWE variant relevant to post-quantum discussion.
Abdalla, Catalano, Fiore, Gay, Ursu — *Multi-Input FE for Inner Products … without Pairings*	CRYPTO 2018	Med-high	§11's MIFE claim — that multi-input builds from single-input without pairings.
Chotard, Dufour Sans, Gay, Phan, Pointcheval — *Decentralized Multi-Client FE for Inner Product*	ASIACRYPT 2018	Med-high	Cross-institution aggregation (§0.7 row 2). PyMIFE's `multiclient/decentralized` likely implements this.
Baltico, Catalano, Fiore, Gay — *Practical FE for Quadratic Functions*	CRYPTO 2017	Med-high	§10.3. PyMIFE's `single/quadratic` module.
Shokri, Stronati, Song, Shmatikov — *Membership Inference Attacks Against ML Models*	IEEE S&P 2017	High	§5.2 method.
Yeom, Giacomelli, Fredrikson, Jha — *Privacy Risk in ML: the Connection to Overfitting*	CSF 2018	High	§5.2's cheap threshold attack baseline.
Milli, Schmidt, Dragan, Hardt — *Model Reconstruction from Model Explanations*	FAT* 2019	Medium	**Directly relevant to §6.** Explanations leak models — likely the closest prior work to your XAI finding.
Jagielski, Carlini, Berthelot, Kurakin, Papernot — *High Accuracy and High Fidelity Extraction of Neural Networks*	USENIX Security 2020	Med-high	The accuracy-versus-fidelity distinction §5.1 now asks you to report.
Homer et al. — *Resolving individuals contributing trace amounts of DNA to…mixtures*	PLoS Genetics 2008	High	Genomic privacy precedent for §0.4.
Cheon, Kim, Kim, Song — *Homomorphic Encryption for Arithmetic of Approximate Numbers* (CKKS)	ASIACRYPT 2017	High	Your FHE baseline in §4.2 (via TenSEAL).
Mitchell et al. — *Model Cards for Model Reporting*	FAT* 2019	High	§7.1's model card format.
Weinstein et al. — *The Cancer Genome Atlas Pan-Cancer Analysis Project*	Nature Genetics 2013	High	Cite for the dataset. Pair with the UCSC Xena paper (Goldman et al., Nat. Biotech. 2020) for the specific resource.

Tier 3 — search these areas rather than specific papers

- **iDASH Privacy & Security Workshop** — annual genomic-privacy competition with published solutions. Directly your domain, a plausible venue, and the best place to see what the field considers state of the art. Search past years' tracks on secure genomic analysis.
- **Secure logistic regression on genomic data** — several HE-based papers, including work by Kim, Song and colleagues in *JMIR Medical Informatics* and *BMC Medical Genomics* around 2018. Your accuracy and latency baselines.
- **Batch effects in TCGA** — needed for §13's cohort-realities item. Search "TCGA batch effect correction" and the ComBat literature.
- **FE surveys** — Boneh, Sahai & Waters, *Functional Encryption: Definitions and Challenges* (TCC 2011) for the definitional frame. High confidence, and useful for your background section.

Practical reading advice

You are four people with eight weeks and you do not need to read all of this. Split Tier 1 two papers each and present to one another — a thirty-minute verbal summary per paper is enough. For Tier 2 read only the abstract, the threat model, and the section your document cites. Keep a shared file with one line per paper: *what it does, what it assumes, and what it means for us.* That file becomes your related-work section, written once and used twice, which is what §9.1 recommends.

One habit worth adopting immediately: whenever this document says "to our knowledge no one has done X," treat it as an **unverified claim assigned to you**, not as a fact. There are four such claims and each is one search away from being settled.

16.5 Check 4 — concrete leads found

Academic databases were not reachable during preparation, but GitHub was, and it yielded three verifiable artifacts.

These are **real, checked repositories**, not recollections — the URLs were fetched. Start here rather than from a blank search box.

Artifact	What was verified	Why it matters and what to check
`hkh112/Enabling-Functional-Encryption-Based-Inference-on-Residual-Architectures` *anonymised paper repo*	README fetched. Uses **RLWE-IPFE** from `fentec-project/IPFE-RLWE`. States: "applying dimensionality constraints to reduce structural reconstruction risk" and "the security analysis focuses on structural reconstruction resistance under dimensionality constraints."	**Read this first, in Week 1.** It is both your best check-4 candidate *and* possible prior art on your own strongest finding. RLWE-IPFE is public-key with no function hiding, and this is a neural-inference pipeline — so the §0.1 pattern is plausibly present. But it also uses *dimensionality as a reconstruction defence*, which is exactly the "dimension is a privacy parameter" result in §1. Either they got there first, in which case cite them and position your contribution as the measurement and the clinical setting, or their treatment is narrower than yours. You must know which before writing.
`fentec-project/neural-network-on-encrypted-data`	README fetched. Demonstrates the **SGP** scheme (links `eprint.iacr.org/2018/206`) evaluating an ML function on encrypted MNIST digits.	Official demo from the FENTEC consortium — the GoFE and CiFEr authors. Second check-4 candidate. Also confirms ePrint **2018/206** exists and is the SGP / "Reading in the Dark" work, so that Tier-1 citation is verified.
`fentec-project/IPFE-RLWE`	README fetched. Ring-LWE IPFE. No mention of function hiding.	The underlying scheme in lead 1. Confirm it is public-key — if so, any deployment giving one party both the public key and a functional key exhibits your pattern.

Also confirmed while checking: GoFE and CiFEr both reference function-hiding schemes in their READMEs, so your proposal's table is right about those two and wrong only about PyMIFE (§2.1).

How to continue from these three. Each names a scheme and probably a paper. Look each paper up on ePrint or Scholar, then use "cited by" and "references" to walk outward — two hops from these three should exhaust the small applied-FE-inference literature. The whole `github.com/fentec-project` organisation is worth browsing; it is the EU FENTEC project's output and the densest concentration of practical FE artifacts that exists.

17. Field notes — things established in discussion

Points that came out of conversation and are not recorded elsewhere in this document. Collected here so nothing depends on a chat log.

17.1 The five-minute test to run first

```
pip install pymife py_ecc

from mife.single.fhiding.ddh import FeDDH
key = FeDDH.generate(4)
c   = FeDDH.encrypt([1,2,3,4], key)
sk  = FeDDH.keygen([5,6,7,8], key)
print(FeDDH.decrypt(c, key.get_public_key(), sk, (0,1000)))   # -> 70
```

1×5 + 2×6 + 3×7 + 4×8 = 70, computed without the library seeing your numbers. If it prints 70, the hard cryptography in your project already works on your machine. Do this before anything else; it takes five minutes and it changes what Workstream A is.

17.2 Perturbation is stronger than this document says

Running `patients_until_recovery.py` on synthetic data shows required patients scaling roughly as `m ~ n · (sigma / tolerance)^2` — **quadratic** in the noise scale, not the "measurable constant factor" stated in §0.1 and §3. On a synthetic 60-dimension model, `sigma`=100 cost 0.19 percentage points of probability accuracy and bought 1.3× lifetime; `sigma`=1000 cost 1.9pp and pushed recovery past 12`n` patients entirely. Confirm on your real weights. If it holds, correct §0.1's control table — perturbation would then be a genuinely useful lever rather than a constant, and that is a better result than the one currently written down.

17.3 The floor — what you have if everything hard fails

If the crypto work stalls, the non-linear experiment fails, and no stretch goal lands, you still have: the extraction attack measured on 33 real models; the two data bugs found and quantified; a quantization–accuracy–calibration study; and an honest map of when this approach works. That is publishable, and it is reachable by roughly Week 4. Everything above it is upside. Knowing where the floor is makes the gates in §8 easy to honour rather than painful.

17.4 Two habits worth more than any document

- **Distinguish measured from estimated.** Every number in this document is one or the other. The estimated ones are where you will be misled — the `ek` size was wrong for one curve, the blinding ceiling was quoted without the joint range, the rescaling bound was a first-round figure. When a number matters, measure it and replace it.
- **Checking that a reference resolves is not the same as checking it is correct.** An automated check that every `§X.Y` points at a section that *exists* passes happily while references point at the wrong section — renumbering silently repoints them at whatever now occupies the slot. Eight such errors survived several revisions of this document that way, including one that would have sent you to redo a check already completed. The check that works: dump every cross-reference beside its target's actual heading and read the list. Ten minutes, and it is the only reliable method. Do this on the paper before you submit, and again after any renumbering.
- **Treat found errors as normal.** Three review rounds found real errors in this plan, including two claims stated confidently and repeated across revisions before anyone caught them. That is the process working, not failing. Keep reviewing during the build, not only before it.

17.5 Unverified claims assigned to you

Wherever this document says "to our knowledge no one has done X," that is a task, not a fact. The list:

- That the lifetime framing (§0.1) is unstated for FE inference.
- That the result-privacy gap (§0.5) is unwritten — *note lead 1 in §16.5 may bear on this.*
- That the `g1^B` delegated-encryption technique (§2.3) is not folklore.
- That dimension-as-privacy-parameter (§1) is novel — *lead 1 explicitly uses dimensionality as a reconstruction defence, so check this one first.*

Each is one search away. Settling them in Week 1 costs two days; discovering them in Week 8 costs the paper.

17.6 If you get stuck on the literature

Anyone helping you who cannot reach academic databases can still work from material you supply — paste an abstract, upload a PDF, or copy a paper's threat-model section, and the "does this exhibit the pattern" question in §16.2 can be answered from that alone. You do not need to hand over a whole paper; the architecture figure and the threat model are enough.

18. Getting it published — the calendar is the problem, not the work

Assumed deadline: **accepted by May 2027**, with build work finishing around October 2026. That leaves roughly seven months of review latency to spend, which is comfortable — enough for one full rejection-and-resubmission cycle. The binding constraint is not whether you can publish but whether you **submit early enough to survive one rejection**.

Backward plan from May 2027 — the only date that matters is the first submission

When	What	Why this date
Aug 2026	Build weeks 1–4. Threat model, literature delta and decision guide drafted *as paper sections*.	These are already prose (§9.1). Writing them now costs nothing extra and removes four sections from the October crunch.
Sept 2026	Build weeks 5–8. Results and figures produced.	Gates 1 and 2 land here; whatever is unfinished at the end of Gate 2 is cut, not rescheduled.
Early Oct 2026	Full draft complete. Internal review by supervisor plus one outside reader.	Three weeks of writing on top of pre-drafted sections is realistic. Do not compress this.
Late Oct / early Nov 2026	**SUBMISSION 1** and preprint posted the same week.	**The critical date.** Everything else is derived from it. Miss it and you lose the ability to absorb a rejection.
Dec 2026 – Jan 2027	Notification.	Typical 6–10 week workshop turnaround. Accepted → done, five months early. Rejected → you have the reviews and time to use them.
Feb 2027	**SUBMISSION 2** if needed, revised against the reviews.	A rejected paper revised with real reviewer feedback is markedly stronger. This is a normal path, not a failure.
Apr 2027	Notification 2.	Lands with a month of margin before your deadline.

The single number to hold onto: submit by early November 2026. Hitting it converts an anxious deadline into a comfortable one, because one rejection stops being fatal. Missing it means your first attempt has to succeed.

18.0 Build the deadline calendar before you build anything else

You cannot backward-plan without knowing when venues actually close, and those dates are the one thing this document cannot supply — they change annually and are outside its knowledge. Spend one hour in Week 1 building a spreadsheet: venue, submission deadline, notification date, page limit, whether a poster or short track exists. Include everything closing between **October 2026 and January 2027**, which is your primary window, and a second tier closing February–March 2027 as fallbacks.

Useful sources: `wikicfp.com` for the general call-for-papers feed; the community-maintained security-conference deadline trackers (search "security conference deadlines"), which list the crypto and privacy venues with countdowns; and the websites of the venue families named in §18.3. Sort by deadline, not by prestige — a venue you can actually hit beats a better one you cannot.

18.1 The question to ask your supervisor first

Departments mean very different things by "published," and the difference is months:

Requirement	Time needed	Implication
Submitted	0 — a receipt	Entirely achievable. Submit anywhere credible and you are done.
Preprint public	1–2 days	arXiv or IACR ePrint. Achievable. Do it regardless of what else you do.
Accepted	6 weeks – 6 months	Feasible only at a workshop, poster track, or regional conference, and only if you submit early.
Presented / in proceedings	4–10 months	Very unlikely inside your degree timeline. Plan around it rather than hoping.

Ask, get the answer in writing, and pick your venue from the answer. This single question is worth more than any amount of extra work, and it costs one email. Many departments accept "preprint posted plus submission under review," which converts a hard problem into an easy one.

18.2 Realistic odds

Two separate probabilities, and conflating them is how people get surprised:

- **Accepted somewhere, eventually** — reasonable, if you execute even two-thirds of this plan, target a workshop rather than a conference, and are willing to move down-market after a rejection. The work is workshop-grade and the §2.1 re-scope makes it more achievable than it was.
- **Accepted before May 2027** — good, provided you submit by early November 2026. The seven-month window absorbs one rejection comfortably. It does not absorb two, and it does not absorb a first submission in February.

So the strategy is: maximise the first, and de-risk the second by making sure a preprint and a submission receipt exist early.

18.3 Venue types, fastest first

Type	Turnaround	Notes
Poster / short paper	4–8 weeks	4 pages instead of 12, lighter review, and counts as a publication at most institutions. **The single best fit for your timeline.** Nearly every conference has one.
Workshops	6–10 weeks	Co-located with large conferences, explicitly welcoming to exploratory and negative results — which is exactly what §9.2 says you have. Best quality-per-week option.
Regional / national conferences	6–12 weeks	Faster and more forgiving. IEEE regional and national events run frequent cycles. Perfectly respectable for a capstone requirement.
Mid-tier conferences	3–5 months	Realistic quality target, unrealistic calendar target. Submit here *after* you have a faster result banked.
Journals	6–18 months	Out of scope for the degree. Consider afterwards if the work holds up.

Venue families worth searching — names from memory, so verify current existence and deadlines yourself, exactly as with §16.4: privacy and applied-crypto workshops co-located with the large security conferences (WPES and WAHC are the two whose scope matches yours most closely); health-informatics venues with bioinformatics tracks such as IEEE BIBM and IEEE HealthCom; applied-security conferences with student and poster tracks such as CODASPY; and the iDASH genomic-privacy workshop, which is the closest thing to a home venue for this exact topic. For a regional option, IEEE national and section conferences run frequently and review quickly.

18.4 What to do, in order

1. **This week:** ask your supervisor what "published" means for your grade, in writing — and confirm the May 2027 date and whether it means accepted, presented, or in proceedings. "In proceedings" can lag acceptance by months and would move your submission date earlier still.
2. **Week 1:** build the deadline calendar (§18.0), then pick a primary venue closing Oct–Nov 2026 and a fallback closing Feb–Mar 2027. Put both in your schedule as hard dates alongside the two build gates.
3. **Throughout:** write as you go. The threat model, the literature delta and the decision guide are already prose and are already paper sections (§9.1). Nothing in this plan should be written for the first time in Week 8.
4. **The moment the draft is coherent:** post to arXiv or IACR ePrint. Free, immediate, citable, timestamped, and it protects your claim to the result. It also satisfies a "preprint" requirement outright.
5. **Submit in November even if it feels early.** A submitted paper can be improved for the next venue; an unsubmitted one cannot be accepted. Reviews are free expert feedback even when they reject, and with a February fallback in hand a rejection costs you nothing but time you have already budgeted. The instinct to polish for another two months is the one that turns a comfortable schedule into a tight one.
6. **Have the 4-page version ready.** Write the short paper first and expand it — not the reverse. It is easier, and it is your fallback.

> **The failure mode to avoid.** A publication requirement creates pressure to overclaim, and overclaiming is the most common reason work at this level is rejected — reviewers check the strong sentences first. The do-not-claim list in §9.3 exists partly for this. Your honest result — a measured lifetime, three demonstrated leaks, a design-space map with real boundaries — is more publishable than an inflated one, because it survives scrutiny. Every claim in this plan that was found to be false across three review rounds was a claim that sounded stronger than the evidence. Do not add more of them under deadline pressure.